CS 245 — Logic and Computation

Vishesh Gupta

University of Waterloo

Contents

1 Introduction: Logic and Computation **1**
- 1.1 Why study logic in computer science? . . . 1
- 1.2 Arguments, premises and conclusions . . . 1
- 1.3 Two views of "follows from" . . . 2
- 1.4 Formal languages: object language vs. metalanguage . . . 3
- 1.5 Translating English into logic . . . 3
- 1.6 Roadmap of the course . . . 3

2 Propositional Logic: Syntax **5**
- 2.1 The alphabet . . . 5
- 2.2 Well-formed formulas . . . 5
  - 2.2.1 Conventions for dropping parentheses . . . 6
- 2.3 Structural induction . . . 6
- 2.4 Unique readability . . . 7
- 2.5 Parse trees . . . 7
- 2.6 Recursive definitions on formulas . . . 8

3 Propositional Logic: Semantics **9**
- 3.1 Truth valuations . . . 9
- 3.2 Satisfiability and validity . . . 10
- 3.3 Logical consequence . . . 10
- 3.4 Logical equivalence . . . 11
- 3.5 Adequate sets of connectives . . . 12
  - 3.5.1 Proving a set is *not* adequate . . . 12
- 3.6 Normal forms . . . 13
  - 3.6.1 Algorithm for CNF by transformation . . . 13

4 Propositional Proof Systems **14**
- 4.1 Natural deduction . . . 14
  - 4.1.1 The rules . . . 15
  - 4.1.2 Example proofs . . . 16
  - 4.1.3 Strategy . . . 17
- 4.2 An axiomatic (Hilbert-style) system . . . 18
- 4.3 Resolution . . . 18

5 Soundness and Completeness of Propositional Logic **20**
5.1 Why we care . . . 20
5.2 Soundness . . . 20
5.3 Completeness . . . 21
5.3.1 The general completeness theorem . . . 22
5.4 Compactness . . . 22
5.5 Decidability . . . 22

6 Predicate Logic: Syntax **24**
6.1 Why propositional logic is not enough . . . 24
6.2 The language . . . 24
6.3 Terms . . . 25
6.4 Formulas . . . 25
6.5 Translating English . . . 25
6.6 Free and bound variables . . . 26
6.7 Substitution . . . 26

7 Predicate Logic: Semantics **28**
7.1 Interpretations . . . 28
7.2 Environments and evaluation . . . 28
7.3 Validity, satisfiability, consequence . . . 29
7.4 Important equivalences . . . 30
7.5 Theories and models . . . 30

8 Natural Deduction for Predicate Logic **32**
8.1 Rules for the quantifiers . . . 32
8.1.1 $\forall$ elimination . . . 32
8.1.2 $\forall$ introduction . . . 32
8.1.3 $\exists$ introduction . . . 32
8.1.4 $\exists$ elimination . . . 32
8.2 Rules for equality . . . 33
8.3 Example proofs . . . 33
8.4 Soundness and completeness . . . 35

9 Peano Arithmetic and Formal Number Theory **37**
9.1 The language of arithmetic . . . 37
9.2 The Peano axioms . . . 37
9.3 Computing with the axioms . . . 38
9.4 Proofs by induction in PA . . . 38
9.5 Forms of induction . . . 39
9.6 Models of PA and the limits of first-order arithmetic . . . 39

10 Program Verification **41**
10.1 The programming language . . . 41
10.2 Hoare triples . . . 41
10.3 Proof rules for partial correctness . . . 42
10.3.1 The assignment rule . . . 42
10.3.2 Proof tableaux . . . 42

10.4 Loops and invariants 43
10.5 Total correctness 44
10.6 Arrays 45

11 Undecidability and Computability **47**
11.1 Turing machines 47
11.2 Decision problems 48
11.3 The halting problem 48
11.4 Reductions 48
11.5 Undecidability of first-order logic 49
11.6 Gödel's incompleteness theorems 49

Chapter 1

Introduction: Logic and Computation

1.1 Why study logic in computer science?

Logic is the study of *correct reasoning*: which conclusions follow from which assumptions, independent of what the statements are actually about. CS 245 treats logic as a mathematical object in its own right. We define precise *languages* (propositional logic and first-order predicate logic), give them a precise *meaning* (semantics), and design *proof systems* that let us derive conclusions by symbol manipulation alone. We then ask how the two sides relate (soundness and completeness) and what the fundamental limits of the approach are (undecidability, incompleteness).

Logic shows up all over computer science:

- **Hardware**: digital circuits are propositional formulas; circuit minimisation is simplification of Boolean formulas.
- **Programming languages**: type systems are logics (the Curry–Howard correspondence: proofs are programs, propositions are types); conditionals and loop guards are formulas.
- **Program verification**: specifications are formulas, and correctness proofs (Hoare logic, Chapter 10) are formal deductions.
- **Databases**: SQL queries are essentially first-order formulas (relational calculus).
- **Artificial intelligence and automated reasoning**: SAT solvers, SMT solvers, theorem provers, model checkers, Prolog.
- **Theory of computation**: the question "is there an algorithm that decides whether a formula is valid?" leads directly to Turing machines and the halting problem (Chapter 11).

1.2 Arguments, premises and conclusions

Definition 1.1 (Proposition)**.** A *proposition* is a declarative sentence that is either true or false (but not both).

Example 1.2. "$2 + 2 = 4$", "Waterloo is in Ontario" and "every even integer greater than 2 is the sum of two primes" are propositions (the last one has an unknown truth value, but it has one). "Close the door!", "Is it raining?" and "$x > 3$" (with x unspecified) are *not* propositions.

Definition 1.3 (Argument)**.** An *argument* consists of a set of propositions called *premises* and one proposition called the *conclusion*. The argument is *valid* if, in every situation in which all the premises are true, the conclusion is also true.

The key word is *every*. Validity depends only on the *form* of the argument, not on whether the premises are actually true.

Example 1.4 (A valid argument)**.** If it is raining, then the ground is wet.
It is raining.
Therefore, the ground is wet.
Writing p for "it is raining" and q for "the ground is wet", the form is

$$p \to q,\ p \ \ \therefore \ \ q.$$

This form (*modus ponens*) is valid no matter what p and q stand for.

Example 1.5 (An invalid argument)**.** If it is raining, then the ground is wet.
The ground is wet.
Therefore, it is raining.
The form $p \to q,\ q \ \therefore\ p$ (*affirming the consequent*) is invalid: take p false and q true (someone ran the sprinkler). Both premises are true and the conclusion is false. One such situation, a *counterexample*, is enough to show invalidity.

Valid vs. sound vs. true

- A *valid* argument can have false premises and a false conclusion: "All cats are fish; all fish fly; therefore all cats fly" is valid.
- An argument is *sound* (in the informal sense) if it is valid *and* all its premises are actually true. Then its conclusion must be true.
- An invalid argument can have a true conclusion; that the conclusion happens to be true does not make the reasoning correct.

(Careful: later we use "soundness" for a property of a *proof system*, which is a different idea.)

1.3 Two views of "follows from"

CS 245 studies two quite different ways of making "the conclusion follows from the premises" precise.

Semantic (model-theoretic). Consider every possible way of assigning meaning to the symbols (every truth valuation, or every interpretation in predicate logic). If the conclusion is true in every one in which all premises are true, we write $\Sigma \models \varphi$ (Σ *semantically entails* φ).

Syntactic (proof-theoretic). Fix a finite set of inference rules. If there is a finite derivation of φ from Σ using only those rules, we write $\Sigma \vdash \varphi$ (φ *is provable from* Σ).

The semantic view is close to our intuition about "truth", but checking infinitely many interpretations is not mechanical. The syntactic view is mechanical (a computer can check a proof), but on its face has nothing to do with truth. The central results of the course are that, for propositional and first-order logic, the two coincide:

$$\Sigma \vdash \varphi \iff \Sigma \models \varphi \qquad \text{(soundness } \Rightarrow\text{, completeness } \Leftarrow\text{)}.$$

1.4 Formal languages: object language vs. metalanguage

When we study a logic we use two languages at once:

- the *object language*: the formal logic being studied, with symbols such as $p, q, \neg, \wedge, \rightarrow, \forall$;
- the *metalanguage*: the (mathematical) English we use to talk *about* the object language, including symbols like $\models$, $\vdash$, "iff" and $\Rightarrow$.

For example, $p \rightarrow q$ is a formula of the object language, while "$\{p \rightarrow q, p\} \models q$" is a statement in the metalanguage that happens to be true. Keeping the two levels apart avoids a lot of confusion, especially when comparing $\rightarrow$ (a connective inside formulas) with $\Rightarrow$ or "implies" (reasoning about formulas).

1.5 Translating English into logic

A recurring skill in the course is translating English sentences into formulas. Some standard translations:

English	Formula
not p; it is not the case that p	$\neg p$
p and q; p but q; p although q	$p \wedge q$
p or q (inclusive)	$p \vee q$
if p then q; q if p; p only if q; p is sufficient for q; q is necessary for p	$p \rightarrow q$
p if and only if q; p exactly when q	$p \leftrightarrow q$
p unless q	$\neg q \rightarrow p$ (equivalently $p \vee q$)
either p or q but not both	$(p \vee q) \wedge \neg(p \wedge q)$
neither p nor q	$\neg p \wedge \neg q$

Pitfall: "only if"

"p only if q" means $p \rightarrow q$, *not* $q \rightarrow p$. "You can graduate only if you pass CS 245" says graduating requires passing; it does not say passing guarantees graduation.

Example 1.6. "If Alice goes to the party and Bob does not, then Carol will be unhappy unless Dave comes." With atoms a, b, c, d:

$$(a \wedge \neg b) \rightarrow (\neg d \rightarrow c).$$

1.6 Roadmap of the course

1. Propositional logic: syntax, semantics, proof systems, soundness and completeness.
2. Predicate (first-order) logic: syntax, semantics, natural deduction, soundness and completeness.
3. Applications: formal arithmetic (Peano axioms), program verification with Hoare logic.
4. Limits: Turing machines, the halting problem, undecidability of first-order validity, Gödel's incompleteness theorems.

Summary

- A proposition is a declarative sentence with a definite truth value.
- An argument is valid iff every situation making all premises true makes the conclusion true; one counterexample shows invalidity.
- Validity is about form, not content; valid $\neq$ true.
- $\models$ (semantic entailment) and $\vdash$ (provability) are the two notions of consequence; soundness and completeness say they agree.
- Distinguish the object language from the metalanguage, and be careful translating "only if" and "unless".

Chapter 2

Propositional Logic: Syntax

Syntax is about which strings of symbols are legal formulas, with no reference yet to truth or meaning.

2.1 The alphabet

The language of propositional logic, $\mathcal{L}^p$, uses the following symbols:

- *propositional variables* (atoms): $p, q, r, \ldots$, possibly with subscripts $p_1, p_2, \ldots$; the set of atoms is written $\mathcal{P}$ (countable);
- *connectives*: $\neg$ (negation), $\wedge$ (conjunction), $\vee$ (disjunction), $\rightarrow$ (implication), $\leftrightarrow$ (biconditional);
- *punctuation*: the parentheses (and).

Some presentations also include the constants $\top$ and $\bot$ as atomic formulas.

An *expression* is any finite string of these symbols. Most expressions, such as $)p\neg \wedge ($, are garbage. We now pick out the meaningful ones.

2.2 Well-formed formulas

Definition 2.1 (Well-formed formula)**.** The set $\mathrm{Form}(\mathcal{L}^p)$ of *well-formed formulas* (wffs, or just formulas) is the smallest set of expressions such that

1. every atom $p \in \mathcal{P}$ is a formula (an *atomic formula*);
2. if A is a formula, then $(\neg A)$ is a formula;
3. if A and B are formulas and $\star \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(A \star B)$ is a formula.

"Smallest" means nothing is a formula unless it is forced to be by finitely many applications of the rules. Equivalently: an expression is a formula iff it has a *formation sequence*, a finite list of expressions each of which is an atom or is obtained from earlier ones by rule 2 or 3, ending with the expression.

Example 2.2. $((p \wedge q) \rightarrow (\neg r))$ is a formula, with formation sequence

$$p, \quad q, \quad r, \quad (p \wedge q), \quad (\neg r), \quad ((p \wedge q) \rightarrow (\neg r)).$$

The expressions $p \wedge q$ (missing outer parentheses), $(p\neg q)$ and $(p\wedge)$ are not formulas under the strict definition.

2.2.1 Conventions for dropping parentheses

Fully parenthesised formulas are hard to read, so we adopt conventions (and always mean the fully parenthesised formula):

- omit the outermost pair of parentheses;
- precedence, from tightest to loosest: $\neg$, $\wedge$, $\vee$, $\to$, $\leftrightarrow$;
- $\to$ associates to the *right*: $p \to q \to r$ means $p \to (q \to r)$;
- $\wedge$ and $\vee$ are usually written associating left; since they are associative semantically the choice rarely matters.

So $\neg p \wedge q \to r \vee s$ abbreviates $(((\neg p) \wedge q) \to (r \vee s))$.

2.3 Structural induction

Because formulas are defined inductively, we prove properties of *all* formulas by induction that follows the definition.

Theorem 2.3 (Principle of structural induction)**.** *Let R be a property of expressions. Suppose*

1. *(base case) every atom has property R;*
2. *(inductive step for $\neg$) if A has R, then $(\neg A)$ has R;*
3. *(inductive step for binary connectives) if A and B have R, then so does $(A \star B)$ for each $\star \in \{\wedge, \vee, \to, \leftrightarrow\}$.*

Then every well-formed formula has property R.

Justification. Let S be the set of formulas with property R. The hypotheses say S contains all atoms and is closed under the formation rules. Since Form is the *smallest* such set, Form $\subseteq S$. (Alternatively, do ordinary strong induction on the length of a formation sequence, or on the number of connectives.) □

Example 2.4 (Balanced parentheses)**.** *Claim*: every formula has the same number of left and right parentheses.

Proof. Let $l(A)$, $r(A)$ be the numbers of left and right parentheses in A.

- Base: an atom p has $l(p) = r(p) = 0$.
- $(\neg A)$: $l = l(A) + 1 = r(A) + 1 = r$ by the induction hypothesis.
- $(A \star B)$: $l = l(A) + l(B) + 1 = r(A) + r(B) + 1 = r$.

By structural induction the claim holds for every formula. □

Example 2.5 (Proper prefixes)**.** *Claim*: every non-empty proper prefix of a formula has strictly more left than right parentheses (in particular, no proper prefix of a formula is a formula).

Sketch. Atoms have no non-empty proper prefixes. For $(\neg A)$, a proper prefix is (, or $(\neg$, or $(\neg A'$ where A' is a prefix of A (a proper prefix, or A itself); in each case the extra leading (is unmatched, using the IH for proper prefixes of A and the balance lemma for A itself. The case $(A \star B)$ is similar.

Example 2.6 (Counting)**.** Let $c(A)$ be the number of occurrences of binary connectives and $a(A)$ the number of occurrences of atoms. If A contains no $\neg$, then $a(A) = c(A) + 1$. Base: $1 = 0 + 1$. Step: $a(A \star B) = a(A) + a(B) = c(A) + 1 + c(B) + 1 = c(A \star B) + 1$.

2.4 Unique readability

Could a formula be built in two different ways, and so have two different meanings? For example, is $(p \wedge q \to r)$ ambiguous? Under the official, fully parenthesised definition, no.

Theorem 2.7 (Unique readability)**.** *Every formula is of exactly one of the following forms:*

1. *an atom;*
2. $(\neg A)$ *for a unique formula* A*;*
3. $(A \star B)$ *for a unique connective* $\star$ *and unique formulas* A*,* B*.*

Proof sketch. The first symbol of a formula is an atom (case 1) or (. If it starts with (¬ then it must come from rule 2, and A is determined by removing the first two and last symbols. Otherwise suppose $(A \star B) = (A' \star' B')$. Then A and A' both start at position 2. If $A \neq A'$, one is a proper prefix of the other, so a proper prefix of a formula is a formula, contradicting the prefix lemma. Hence $A = A'$, so $\star = \star'$ and $B = B'$. □

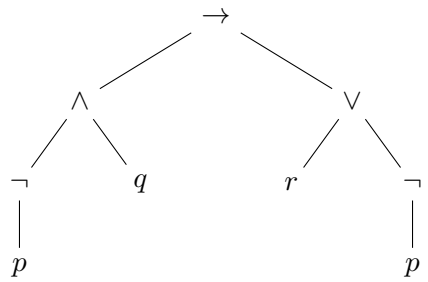
Unique readability is what lets us define functions on formulas by *structural recursion* (e.g. the truth value of a formula, its height, its set of subformulas) and know they are well defined.

Definition 2.8 (Main connective, subformula)**.** In case 2 or 3 above, ¬ (respectively $\star$) is the *main connective* of the formula. The set of *subformulas* is defined recursively: $\mathrm{sub}(p) = \{p\}$, $\mathrm{sub}((\neg A)) = \{(\neg A)\} \cup \mathrm{sub}(A)$, and $\mathrm{sub}((A \star B)) = \{(A \star B)\} \cup \mathrm{sub}(A) \cup \mathrm{sub}(B)$.

2.5 Parse trees

A formula can be drawn as a *parse tree*: leaves are atoms, each internal node is labelled with a connective, ¬ nodes have one child and binary nodes have two. The root is the main connective. Unique readability says each formula has exactly one parse tree.

Example 2.9. The parse tree of $\neg p \wedge q \to (r \vee \neg p)$, i.e. $(((\neg p) \wedge q) \to (r \vee (\neg p)))$:



Every subtree corresponds to a subformula. The *height* of the formula is the height of the tree (here 3, counting edges on the longest root-to-leaf path).

> Checking whether a string is a well-formed formula is a parsing problem, like in a compiler. A simple algorithm: if the string is an atom, accept; if it starts with (¬ and ends with), recurse on the middle; otherwise, scan from the second symbol keeping a parenthesis counter and find the first point where the counter returns to 0 after a complete formula; the next symbol must be a binary connective, and recurse on both sides. The prefix lemma guarantees this split point is the only possible one.

2.6 Recursive definitions on formulas

Some standard functions defined by structural recursion:

$$\begin{aligned} \mathrm{height}(p) &= 0, & \mathrm{height}(\neg A) &= 1 + \mathrm{height}(A), \\ & & \mathrm{height}(A \star B) &= 1 + \max(\mathrm{height}(A), \mathrm{height}(B)). \\ \mathrm{atoms}(p) &= \{p\}, & \mathrm{atoms}(\neg A) &= \mathrm{atoms}(A), \\ & & \mathrm{atoms}(A \star B) &= \mathrm{atoms}(A) \cup \mathrm{atoms}(B). \end{aligned}$$

Proofs about such functions are then by structural induction. For instance, one can show a formula of height h has at most $2^{h+1} - 1$ symbols excluding parentheses.

Summary

- Formulas are the smallest set containing the atoms and closed under $(\neg A)$ and $(A \star B)$.
- Structural induction: prove for atoms, then show each formation rule preserves the property.
- Lemmas: parentheses balance; proper prefixes have excess left parentheses.
- Unique readability: every formula decomposes in exactly one way, so it has one parse tree and a unique main connective, and recursive definitions are well defined.
- Precedence conventions ($\neg, \wedge, \vee, \rightarrow, \leftrightarrow$; $\rightarrow$ is right-associative) are only abbreviations.

Chapter 3

Propositional Logic: Semantics

3.1 Truth valuations

Definition 3.1 (Truth valuation)**.** A *truth valuation* (or assignment) is a function $t : \mathcal{P} \to \{\mathsf{T}, \mathsf{F}\}$ assigning a truth value to every atom. (Some texts write $1, 0$ instead of T, F.)

By unique readability, t extends uniquely to a function $A \mapsto A^t$ on all formulas, defined by structural recursion:

$$\begin{aligned} p^t &= t(p), \\ (\neg A)^t &= \mathsf{T} \text{ iff } A^t = \mathsf{F}, \\ (A \wedge B)^t &= \mathsf{T} \text{ iff } A^t = \mathsf{T} \text{ and } B^t = \mathsf{T}, \\ (A \vee B)^t &= \mathsf{T} \text{ iff } A^t = \mathsf{T} \text{ or } B^t = \mathsf{T}, \\ (A \to B)^t &= \mathsf{T} \text{ iff } A^t = \mathsf{F} \text{ or } B^t = \mathsf{T}, \\ (A \leftrightarrow B)^t &= \mathsf{T} \text{ iff } A^t = B^t. \end{aligned}$$

As truth tables:

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \to B$	$A \leftrightarrow B$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Material implication

$A \to B$ is false *only* when A is true and B is false. In particular, an implication with a false antecedent is (vacuously) true: "if $1 = 2$ then the moon is cheese" is true. This is the convention that makes logic work smoothly, even if it sounds odd in English.

Lemma 3.2 (Relevance)**.** *A^t depends only on the values t assigns to atoms occurring in A: if $t_1(p) = t_2(p)$ for every $p \in \mathrm{atoms}(A)$, then $A^{t_1} = A^{t_2}$.*

Proof. Structural induction on A, using the recursive definition at each step. □

So a formula with n distinct atoms has only 2^n relevantly different valuations, which is what makes truth tables finite.

Example 3.3. Truth table for $(p \to q) \wedge (q \to r) \to (p \to r)$:

p	q	r	$p \to q$	$q \to r$	$p \to r$	whole
T	T	T	T	T	T	T
T	T	F	T	F	F	T
T	F	T	F	T	T	T
T	F	F	F	T	F	T
F	T	T	T	T	T	T
F	T	F	T	F	T	T
F	F	T	T	T	T	T
F	F	F	T	T	T	T

The last column is all T: the formula is a tautology (transitivity of $\to$).

3.2 Satisfiability and validity

Definition 3.4. Let A be a formula and Σ a set of formulas.

- t *satisfies* A if $A^t = \mathsf{T}$; t satisfies Σ if it satisfies every formula in Σ (write $\Sigma^t = \mathsf{T}$).
- A is a *tautology* (valid) if every valuation satisfies it.
- A is a *contradiction* (unsatisfiable) if no valuation satisfies it.
- A is *satisfiable* if some valuation satisfies it; a set Σ is satisfiable if some single valuation satisfies all of its members.
- A formula that is satisfiable but not a tautology is *contingent*.
- $p \vee \neg p$ is a tautology (law of excluded middle);
- $p \wedge \neg p$ is a contradiction;
- $p \to q$ is contingent.

Note: A is a tautology iff $\neg A$ is a contradiction (unsatisfiable).

Satisfiable sets

A set can consist of satisfiable formulas and still be unsatisfiable: $\{p,\ p \to q,\ \neg q\}$ has each member satisfiable, but no single valuation satisfies all three. The empty set is satisfiable (every valuation satisfies it vacuously).

3.3 Logical consequence

Definition 3.5 (Tautological consequence)**.** $\Sigma \models A$ (A *is a tautological consequence of* Σ) if every valuation that satisfies Σ also satisfies A. We write $\models A$ for $\emptyset \models A$; this holds iff A is a tautology.

To show $\Sigma \models A$, argue that any t with $\Sigma^t = \mathsf{T}$ has $A^t = \mathsf{T}$ (or check a truth table restricted to rows where all premises are true). To show $\Sigma \not\models A$, exhibit one valuation making all of Σ true and A false.

Theorem 3.6 (Useful facts)**.** *For formulas A, B and sets Σ:*

1. *$\Sigma \models A$ iff $\Sigma \cup \{\neg A\}$ is unsatisfiable.*
2. *(Deduction) $\Sigma \cup \{A\} \models B$ iff $\Sigma \models A \to B$.*
3. *$\{A_1, \ldots, A_n\} \models B$ iff $\models (A_1 \wedge \cdots \wedge A_n) \to B$.*
4. *(Monotonicity) If $\Sigma \models A$ and $\Sigma \subseteq \Sigma'$ then $\Sigma' \models A$.*
5. *If Σ is unsatisfiable, then $\Sigma \models A$ for every A.*

Proof of 2. ($\Rightarrow$) Let t satisfy Σ. If $A^t = \mathsf{F}$, then $(A \to B)^t = \mathsf{T}$. If $A^t = \mathsf{T}$, then t satisfies $\Sigma \cup \{A\}$ so $B^t = \mathsf{T}$ and $(A \to B)^t = \mathsf{T}$. ($\Leftarrow$) Let t satisfy $\Sigma \cup \{A\}$. Then $(A \to B)^t = \mathsf{T}$ and $A^t = \mathsf{T}$, so $B^t = \mathsf{T}$. □

Fact 1 is the basis of *refutation* methods such as resolution: to prove A from Σ, show $\Sigma \cup \{\neg A\}$ has no model.

3.4 Logical equivalence

Definition 3.7. A and B are *logically equivalent*, written $A \equiv B$ (or $A \dashv\vdash B$), if $A^t = B^t$ for every valuation t. Equivalently, $A \leftrightarrow B$ is a tautology, or both $A \models B$ and $B \models A$.

Standard equivalences

Double negation	$\neg\neg A \equiv A$
De Morgan	$\neg(A \wedge B) \equiv \neg A \vee \neg B, \quad \neg(A \vee B) \equiv \neg A \wedge \neg B$
Commutativity	$A \wedge B \equiv B \wedge A, \quad A \vee B \equiv B \vee A$
Associativity	$(A \wedge B) \wedge C \equiv A \wedge (B \wedge C)$, similarly for $\vee$
Distributivity	$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$
	$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
Idempotence	$A \wedge A \equiv A, \quad A \vee A \equiv A$
Absorption	$A \wedge (A \vee B) \equiv A, \quad A \vee (A \wedge B) \equiv A$
Implication	$A \to B \equiv \neg A \vee B$
Contrapositive	$A \to B \equiv \neg B \to \neg A$
Biconditional	$A \leftrightarrow B \equiv (A \to B) \wedge (B \to A)$
Exportation	$(A \wedge B) \to C \equiv A \to (B \to C)$
Simplification	$A \vee \top \equiv \top$, $A \wedge \top \equiv A$, $A \vee \bot \equiv A$, $A \wedge \bot \equiv \bot$
Complement	$A \vee \neg A \equiv \top, \quad A \wedge \neg A \equiv \bot$

Theorem 3.8 (Replaceability / substitution)**.** 1. *If $B \equiv C$ and A' results from A by replacing one or more occurrences of the subformula B by C, then $A \equiv A'$.*

2. *If A is a tautology and A' results from replacing every occurrence of an atom p in A by a formula B, then A' is a tautology.*

Together these let us do *algebraic* proofs of equivalence.

Example 3.9. Show $(p \to q) \wedge (p \to r) \equiv p \to (q \wedge r)$:

$$\begin{aligned} (p \to q) \wedge (p \to r) &\equiv (\neg p \vee q) \wedge (\neg p \vee r) && \text{implication} \\ &\equiv \neg p \vee (q \wedge r) && \text{distributivity} \\ &\equiv p \to (q \wedge r) && \text{implication.} \end{aligned}$$

Example 3.10. Simplify $\neg(p \vee \neg q) \vee (\neg p \wedge \neg q)$:

$$\begin{aligned} &\equiv (\neg p \wedge \neg\neg q) \vee (\neg p \wedge \neg q) && \text{De Morgan} \\ &\equiv (\neg p \wedge q) \vee (\neg p \wedge \neg q) && \text{double negation} \\ &\equiv \neg p \wedge (q \vee \neg q) && \text{distributivity} \\ &\equiv \neg p \wedge \top \equiv \neg p. \end{aligned}$$

3.5 Adequate sets of connectives

Every formula with atoms $p_1, \ldots, p_n$ defines a *truth function* $\{\mathsf{T}, \mathsf{F}\}^n \to \{\mathsf{T}, \mathsf{F}\}$. There are 2^{2^n} truth functions of n arguments (16 binary ones).

Definition 3.11. A set of connectives is *adequate* (functionally complete) if every truth function of every arity $n \geq 1$ is represented by some formula using only those connectives.

Theorem 3.12. *$\{\neg, \wedge, \vee\}$ is adequate.*

Proof. Let f be an n-ary truth function. If f is always F, use $p_1 \wedge \neg p_1$. Otherwise, for each row $(v_1, \ldots, v_n)$ with $f(v_1, \ldots, v_n) = \mathsf{T}$ form the conjunction $\ell_1 \wedge \cdots \wedge \ell_n$ where $\ell_i = p_i$ if $v_i = \mathsf{T}$ and $\ell_i = \neg p_i$ if $v_i = \mathsf{F}$. This conjunction is true exactly on that row. The disjunction of all such conjunctions is true exactly on the rows where f is T. □

Corollary 3.13. *$\{\neg, \wedge\}$, $\{\neg, \vee\}$ and $\{\neg, \to\}$ are adequate.*

Proof. It suffices to express the missing connectives of an adequate set: $A \vee B \equiv \neg(\neg A \wedge \neg B)$; $A \wedge B \equiv \neg(\neg A \vee \neg B)$; $A \vee B \equiv \neg A \to B$ (so $\{\neg, \to\}$ gives $\vee$, hence reduces to $\{\neg, \vee\}$). □

The single connectives NAND ($A \uparrow B \equiv \neg(A \wedge B)$, the Sheffer stroke) and NOR ($A \downarrow B \equiv \neg(A \vee B)$) are each adequate on their own: $\neg A \equiv A \uparrow A$ and $A \wedge B \equiv (A \uparrow B) \uparrow (A \uparrow B)$.

3.5.1 Proving a set is *not* adequate

Find a property preserved by all formulas built from the set that some truth function lacks, and prove it by structural induction.

Example 3.14. $\{\wedge, \vee\}$ is not adequate. *Claim*: every formula A built from atoms with only $\wedge, \vee$ satisfies $A^t = \mathsf{T}$ when t makes every atom true. Base: trivially. Step: $\mathsf{T} \wedge \mathsf{T} = \mathsf{T}$, $\mathsf{T} \vee \mathsf{T} = \mathsf{T}$. So $\neg p$ cannot be expressed.

Example 3.15. $\{\to\}$ is not adequate: every formula built from $\to$ alone is true under the all-T valuation (since $\mathsf{T} \to \mathsf{T} = \mathsf{T}$), so again $\neg p$ is not expressible. Similarly $\{\neg, \leftrightarrow\}$ is not adequate: one can show every such formula in two variables has an even number of Ts in its truth table, but $p \wedge q$ has one.

3.6 Normal forms

Definition 3.16. A *literal* is an atom p or its negation $\neg p$. A *clause* (disjunctive clause) is a disjunction of literals; a *conjunctive clause* is a conjunction of literals.

- A formula is in *disjunctive normal form* (DNF) if it is a disjunction of conjunctions of literals: $\bigvee_i \bigwedge_j \ell_{ij}$.
- A formula is in *conjunctive normal form* (CNF) if it is a conjunction of disjunctions of literals: $\bigwedge_i \bigvee_j \ell_{ij}$.

The proof of adequacy above constructs a DNF directly from the truth table. Dually, a CNF can be read off the rows where f is F: for each such row include the clause that is false exactly on that row (flip each literal).

Theorem 3.17. *Every formula is equivalent to one in DNF and to one in CNF.*

3.6.1 Algorithm for CNF by transformation

1. Eliminate $\leftrightarrow$ and $\rightarrow$: $A \leftrightarrow B \equiv (\neg A \vee B) \wedge (A \vee \neg B)$ and $A \rightarrow B \equiv \neg A \vee B$.
2. Push negations inward (to atoms) using De Morgan and double negation (*negation normal form*).
3. Distribute $\vee$ over $\wedge$: $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$, until no $\wedge$ is inside a $\vee$.

For DNF, distribute $\wedge$ over $\vee$ in the last step instead.

Example 3.18. Convert $(p \rightarrow q) \rightarrow r$ to CNF:

$$(p \rightarrow q) \rightarrow r \equiv \neg(\neg p \vee q) \vee r \equiv (p \wedge \neg q) \vee r \equiv (p \vee r) \wedge (\neg q \vee r).$$

In DNF it is $(p \wedge \neg q) \vee r$.

> CNF conversion by distribution can blow up exponentially: $(p_1 \wedge q_1) \vee \cdots \vee (p_n \wedge q_n)$ has a CNF with 2^n clauses. In practice SAT solvers use the *Tseitin transformation*, which introduces fresh atoms and produces an *equisatisfiable* (not equivalent) CNF of linear size. Deciding satisfiability of a CNF formula (SAT) is NP-complete; deciding it for DNF is trivial (some conjunct must not contain both p and $\neg p$).

Summary

- A valuation fixes atoms; truth values of compound formulas follow by recursion.
- Tautology: true under all valuations; contradiction: none; satisfiable: some.
- $\Sigma \models A$: every valuation satisfying Σ satisfies A; equivalently $\Sigma \cup \{\neg A\}$ is unsatisfiable.
- $A \equiv B$ iff they agree on every valuation; use known equivalences and replaceability for algebraic proofs.
- $\{\neg, \wedge, \vee\}$, $\{\neg, \wedge\}$, $\{\neg, \rightarrow\}$, $\{\uparrow\}$, $\{\downarrow\}$ are adequate; show non-adequacy with an invariant proved by structural induction.
- Every formula has equivalent CNF and DNF forms (from truth tables or by rewriting).

Chapter 4

Propositional Proof Systems

Truth tables decide $\Sigma \models A$ for finite Σ, but they grow as 2^n and do not generalise to predicate logic (infinitely many interpretations). A *proof system* gives purely syntactic rules for deriving formulas. We study three: natural deduction (the main system in the course), a Hilbert-style axiomatic system, and resolution.

4.1 Natural deduction

Natural deduction (following Huth and Ryan, *Logic in Computer Science*) has, for each connective, *introduction* rules (how to prove a formula with that main connective) and *elimination* rules (how to use one).

Definition 4.1 (Sequent)**.** A *sequent* $\varphi_1, \ldots, \varphi_n \vdash \psi$ asserts that ψ can be derived from the premises $\varphi_1, \ldots, \varphi_n$ using the rules. It is *valid* if such a proof exists. If $\vdash \psi$ (no premises) is valid, ψ is a *theorem*.

A proof is a numbered list of lines. Each line holds one formula and a justification: *premise*, *assumption* (which opens a box), or a rule name with the line numbers it uses. A *box* (drawn as a vertical bar below) is a sub-proof that begins with an assumption; formulas inside a box may not be used after the box is closed. The box itself (a range of lines i–j) can be cited by the rules that discharge assumptions.

4.1.1 The rules

	Introduction	Elimination
$\wedge$	$\dfrac{\varphi \quad \psi}{\varphi \wedge \psi}\ \wedge i$	$\dfrac{\varphi \wedge \psi}{\varphi}\ \wedge e_1$ $\quad$ $\dfrac{\varphi \wedge \psi}{\psi}\ \wedge e_2$
$\vee$	$\dfrac{\varphi}{\varphi \vee \psi}\ \vee i_1$ $\quad$ $\dfrac{\psi}{\varphi \vee \psi}\ \vee i_2$	$\dfrac{\varphi \vee \psi \quad \boxed{\begin{array}{c}\varphi\\ \vdots\\ \chi\end{array}} \quad \boxed{\begin{array}{c}\psi\\ \vdots\\ \chi\end{array}}}{\chi}\ \vee e$
$\rightarrow$	$\dfrac{\boxed{\begin{array}{c}\varphi\\ \vdots\\ \psi\end{array}}}{\varphi \rightarrow \psi}\ \rightarrow i$	$\dfrac{\varphi \quad \varphi \rightarrow \psi}{\psi}\ \rightarrow e$
$\neg$	$\dfrac{\boxed{\begin{array}{c}\varphi\\ \vdots\\ \bot\end{array}}}{\neg\varphi}\ \neg i$	$\dfrac{\varphi \quad \neg\varphi}{\bot}\ \neg e$
$\bot$		$\dfrac{\bot}{\varphi}\ \bot e$
$\neg\neg$		$\dfrac{\neg\neg\varphi}{\varphi}\ \neg\neg e$

Useful *derived* rules (each provable from the basic ones, so they may be used as shortcuts):

$$\frac{\varphi \rightarrow \psi \quad \neg\psi}{\neg\varphi}\ \text{MT} \qquad \frac{\varphi}{\neg\neg\varphi}\ \neg\neg i \qquad \frac{\boxed{\begin{array}{c}\neg\varphi\\ \vdots\\ \bot\end{array}}}{\varphi}\ \text{PBC} \qquad \frac{}{\varphi \vee \neg\varphi}\ \text{LEM}$$

PBC is *proof by contradiction*. There is also a "copy" rule allowing an earlier visible formula to be repeated.

Box discipline

- Every box opens with an *assumption* and must be closed before the proof ends.
- After a box closes, no line inside it may be cited individually; only the whole box can be cited by $\to i$, $\neg i$, $\vee e$ or PBC.
- Inside a box you may use any line above it that is still visible (premises, or lines in enclosing open boxes).

4.1.2 Example proofs

Example 4.2 ($p \wedge q \vdash q \wedge p$)**.**

1	$p \wedge q$	premise
2	q	$\wedge e_2$ 1
3	p	$\wedge e_1$ 1
4	$q \wedge p$	$\wedge i$ 2, 3

Example 4.3 ($p \to q,\ q \to r \vdash p \to r$)**.**

1	$p \to q$	premise
2	$q \to r$	premise
3	$\vert\ p$	assumption
4	$\vert\ q$	$\to e$ 3, 1
5	$\vert\ r$	$\to e$ 4, 2
6	$p \to r$	$\to i$ 3–5

Example 4.4 (Modus tollens: $p \to q,\ \neg q \vdash \neg p$)**.**

1	$p \to q$	premise
2	$\neg q$	premise
3	$\vert\ p$	assumption
4	$\vert\ q$	$\to e$ 3, 1
5	$\vert\ \bot$	$\neg e$ 4, 2
6	$\neg p$	$\neg i$ 3–5

Example 4.5 (Disjunctive syllogism: $p \vee q,\ \neg p \vdash q$)**.**

1	$p \vee q$	premise
2	$\neg p$	premise
3	$\vert\ p$	assumption
4	$\vert\ \bot$	$\neg e$ 3, 2
5	$\vert\ q$	$\bot e$ 4
6	$\vert\ q$	assumption
7	q	$\vee e$ 1, 3–5, 6–6

Lines 3–5 and 6–6 are two separate boxes (one per disjunct).

Example 4.6 (Law of excluded middle: $\vdash p \vee \neg p$)**.**

1	$\neg(p \vee \neg p)$	assumption
2	$\quad p$	assumption
3	$\quad p \vee \neg p$	$\vee i_1$ 2
4	$\quad \bot$	$\neg e$ 3, 1
5	$\neg p$	$\neg i$ 2–4
6	$p \vee \neg p$	$\vee i_2$ 5
7	$\bot$	$\neg e$ 6, 1
8	$\neg\neg(p \vee \neg p)$	$\neg i$ 1–7
9	$p \vee \neg p$	$\neg\neg e$ 8

Example 4.7 (De Morgan: $\neg(p \vee q) \vdash \neg p \wedge \neg q$)**.**

1	$\neg(p \vee q)$	premise
2	$\quad p$	assumption
3	$\quad p \vee q$	$\vee i_1$ 2
4	$\quad \bot$	$\neg e$ 3, 1
5	$\neg p$	$\neg i$ 2–4
6	$\quad q$	assumption
7	$\quad p \vee q$	$\vee i_2$ 6
8	$\quad \bot$	$\neg e$ 7, 1
9	$\neg q$	$\neg i$ 6–8
10	$\neg p \wedge \neg q$	$\wedge i$ 5, 9

Example 4.8 (Exportation: $p \wedge q \to r \vdash p \to (q \to r)$)**.**

1	$p \wedge q \to r$	premise
2	$\quad p$	assumption
3	$\quad\quad q$	assumption
4	$\quad\quad p \wedge q$	$\wedge i$ 2, 3
5	$\quad\quad r$	$\to e$ 4, 1
6	$\quad q \to r$	$\to i$ 3–5
7	$p \to (q \to r)$	$\to i$ 2–6

4.1.3 Strategy

- Work *backwards* from the goal using its main connective: to prove $\varphi \to \psi$, assume φ and aim for ψ; to prove $\neg\varphi$, assume φ and aim for $\bot$; to prove $\varphi \wedge \psi$, prove both.
- Work *forwards* from premises with elimination rules: split conjunctions, apply $\to e$, and case-split disjunctions with $\vee e$.
- If stuck (especially for a goal that is an atom or a disjunction), try PBC: assume $\neg$(goal) and derive $\bot$.

The rules $\neg\neg e$ (equivalently PBC or LEM) are what make this *classical* logic. Dropping them gives *intuitionistic* logic, where a proof of $\varphi \vee \psi$ must actually prove one of the disjuncts. Intuitionistic proofs correspond to programs (Curry–Howard).

Some terms of CS 245 (following Lu's course notes) used an equivalent "formal deduction" system written with sequents $\Sigma \vdash A$ and rules named (Ref) $A \vdash A$; (+) add premises; ($\neg-$) if $\Sigma, \neg A \vdash B$ and $\Sigma, \neg A \vdash \neg B$ then $\Sigma \vdash A$; ($\to -$), ($\to +$), ($\wedge\pm$), ($\vee\pm$), ($\leftrightarrow \pm$) and ($\in$) $A \in \Sigma \Rightarrow \Sigma \vdash A$. The proofs are the same in spirit; only the bookkeeping differs.

4.2 An axiomatic (Hilbert-style) system

A Hilbert system has many axioms and very few rules. A common one uses only $\neg$ and $\to$ (an adequate set). For any formulas φ, ψ, χ, every instance of these schemes is an axiom:

$$\begin{aligned}
&\text{(A1)} \quad \varphi \to (\psi \to \varphi) \\
&\text{(A2)} \quad (\varphi \to (\psi \to \chi)) \to ((\varphi \to \psi) \to (\varphi \to \chi)) \\
&\text{(A3)} \quad (\neg\psi \to \neg\varphi) \to (\varphi \to \psi)
\end{aligned}$$

The only rule is *modus ponens* (MP): from φ and $\varphi \to \psi$ infer ψ.

A *proof of φ from Σ* is a finite sequence of formulas, each an axiom, a member of Σ, or obtained by MP from two earlier ones, ending in φ.

Example 4.9 ($\vdash \varphi \to \varphi$)**.**

1	$(\varphi \to ((\varphi \to \varphi) \to \varphi)) \to ((\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi))$	A2
2	$\varphi \to ((\varphi \to \varphi) \to \varphi)$	A1
3	$(\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi)$	MP 2, 1
4	$\varphi \to (\varphi \to \varphi)$	A1
5	$\varphi \to \varphi$	MP 4, 3

Even this trivial theorem is awkward, which is why one proves the deduction theorem.

Theorem 4.10 (Deduction theorem)**.** $\Sigma \cup \{\varphi\} \vdash \psi$ *iff* $\Sigma \vdash \varphi \to \psi$.

Proof sketch. ($\Leftarrow$) Add φ and apply MP. ($\Rightarrow$) Induction on the length of a proof $\psi_1, \ldots, \psi_n = \psi$ from $\Sigma \cup \{\varphi\}$, showing $\Sigma \vdash \varphi \to \psi_i$ for each i: if ψ_i is an axiom or in Σ, use ψ_i, A1 ($\psi_i \to (\varphi \to \psi_i)$), and MP; if $\psi_i = \varphi$, use $\vdash \varphi \to \varphi$; if ψ_i comes by MP from ψ_j and $\psi_k = \psi_j \to \psi_i$, then by IH we have $\varphi \to \psi_j$ and $\varphi \to (\psi_j \to \psi_i)$, and two uses of A2 with MP give $\varphi \to \psi_i$. □

The deduction theorem plays the role of $\to i$ in natural deduction. Natural deduction and this Hilbert system prove exactly the same sequents (for formulas over $\neg, \to$).

4.3 Resolution

Resolution is a refutation system designed for machines. It works on formulas in CNF.

Definition 4.11. Represent a clause as a *set* of literals, and a CNF formula as a set of clauses. The empty clause, written □, is unsatisfiable.

Definition 4.12 (Resolution rule)**.** From clauses $C \cup \{p\}$ and $D \cup \{\neg p\}$, infer the *resolvent* $C \cup D$:

$$\frac{C \vee p \qquad D \vee \neg p}{C \vee D}.$$

The rule is sound: any valuation satisfying both premises satisfies $C \cup D$ (if p is true then D must be true; if p is false then C must be).

Theorem 4.13 (Soundness and refutation completeness of resolution)**.** *A finite set of clauses S is unsatisfiable iff* □ *can be derived from S by repeated resolution.*

Method. To show $\Sigma \models \varphi$: convert each formula in $\Sigma \cup \{\neg\varphi\}$ to CNF, collect all the clauses, and derive $\square$.

Example 4.14. Show $\{p \to q,\ q \to r,\ p\} \models r$. Clauses:

1. $\{\neg p, q\}$ 2. $\{\neg q, r\}$ 3. $\{p\}$ 4. $\{\neg r\}$ (negated goal)

5	$\{q\}$	resolve 1, 3 on p
6	$\{r\}$	resolve 5, 2 on q
7	$\square$	resolve 6, 4 on r

Example 4.15. Show $\{p \vee q,\ p \to r,\ q \to r\} \models r$. Clauses: $\{p, q\}$, $\{\neg p, r\}$, $\{\neg q, r\}$, $\{\neg r\}$. Resolve $\{\neg p, r\}$ with $\{\neg r\}$ to get $\{\neg p\}$; $\{\neg q, r\}$ with $\{\neg r\}$ to get $\{\neg q\}$; $\{p, q\}$ with $\{\neg p\}$ to get $\{q\}$; $\{q\}$ with $\{\neg q\}$ to get $\square$.

Resolution pitfalls

- Resolve on *one* complementary pair at a time: from $\{p, q\}$ and $\{\neg p, \neg q\}$ you may *not* conclude $\square$ (the resolvents are tautologies like $\{q, \neg q\}$, which can be discarded).
- Duplicate literals merge because clauses are sets.
- Resolution is refutation complete, not complete for direct derivation: it cannot derive $p \vee q$ from $\{p\}$, but it can refute $\{p, \neg p \wedge \neg q\}$.

The Davis–Putnam procedure eliminates one atom at a time by forming all resolvents on it. Modern SAT solvers (DPLL/CDCL) combine backtracking search, unit propagation and learned clauses, which are resolvents.

Summary

- Natural deduction: intro/elim rules per connective, boxes for assumptions; $\to i$, $\neg i$, $\vee e$, PBC discharge assumptions.
- Derived rules: MT, $\neg\neg i$, PBC, LEM.
- Hilbert system: axioms A1–A3 plus modus ponens; the deduction theorem recovers $\to i$.
- Resolution: put $\Sigma \cup \{\neg\varphi\}$ in clause form and derive the empty clause; sound and refutation complete.

Chapter 5

Soundness and Completeness of Propositional Logic

We now have two notions of consequence: $\Sigma \models \varphi$ (semantic, via truth valuations) and $\Sigma \vdash \varphi$ (syntactic, via natural deduction). This chapter proves they coincide.

The two theorems

Soundness: if $\Sigma \vdash \varphi$ then $\Sigma \models \varphi$. (*Everything provable is true*: the rules never lead us astray.)

Completeness: if $\Sigma \models \varphi$ then $\Sigma \vdash \varphi$. (*Everything true is provable*: the rules are strong enough.)

5.1 Why we care

- Soundness gives a way to show something is *not* provable: if we find a valuation making Σ true and φ false, then $\Sigma \not\models \varphi$, so $\Sigma \nvdash \varphi$. Proving non-provability directly ("no proof exists") would otherwise be very hard.
- Completeness says that whenever a truth table confirms an entailment, a formal proof exists, so we may search for one with confidence.

5.2 Soundness

Theorem 5.1 (Soundness)**.** *If $\varphi_1, \ldots, \varphi_n \vdash \psi$ is valid (has a natural deduction proof) then $\varphi_1, \ldots, \varphi_n \models \psi$.*

Proof. We prove a stronger statement by (strong) induction on the line number k of a proof:

$M(k)$*: if the formula on line k is χ, and $\alpha_1, \ldots, \alpha_m$ are the assumptions of the boxes open at line k, then $\varphi_1, \ldots, \varphi_n, \alpha_1, \ldots, \alpha_m \models \chi$.*

Applying $M(k)$ to the last line (where no boxes are open) gives the theorem. Assume $M(j)$ for all $j < k$ and consider the justification of line k.

- *Premise*: $\chi = \varphi_i$, and trivially $\varphi_1, \ldots, \varphi_n \models \varphi_i$.
- *Assumption*: $\chi = \alpha_m$ is itself one of the open assumptions, so the entailment is trivial.

- $\wedge i$ from lines $j_1, j_2 < k$ with $\chi = \chi_1 \wedge \chi_2$: lines j_1, j_2 are visible from k, so their open assumptions are among those at k. By IH any valuation satisfying the premises and open assumptions makes χ_1 and χ_2 true, hence χ true. (Monotonicity handles the extra assumptions.)
- $\to e$ from χ_1 and $\chi_1 \to \chi$: any valuation making both true makes χ true.
- $\to i$ with $\chi = \chi_1 \to \chi_2$ from the box i–j: by IH applied to line j, φ's, α's, $\chi_1 \models \chi_2$. By the semantic deduction theorem, φ's, α's $\models \chi_1 \to \chi_2$.
- $\neg i$ from a box $\chi_1 \ \ldots \ \bot$: by IH the premises, open assumptions and χ_1 entail $\bot$, i.e. no valuation satisfies them all; so every valuation satisfying the premises and open assumptions makes χ_1 false, and $\neg\chi_1$ true.
- $\vee e$: let t satisfy the premises and open assumptions. By IH $(\chi_1 \vee \chi_2)^t = \mathsf{T}$. If $\chi_1^t = \mathsf{T}$, the IH on the last line of the first box gives $\chi^t = \mathsf{T}$; otherwise $\chi_2^t = \mathsf{T}$ and the second box gives it.
- The remaining rules ($\wedge e$, $\vee i$, $\neg e$, $\bot e$, $\neg\neg e$) are similar: in each case the conclusion is true under every valuation that makes the cited lines true. ($\bot e$ holds vacuously: no valuation makes $\bot$ true.)

□

Example 5.2 (Using soundness)**.** Is $p \to q \vdash q \to p$ valid? Take $t(p) = \mathsf{F}$, $t(q) = \mathsf{T}$: then $(p \to q)^t = \mathsf{T}$ but $(q \to p)^t = \mathsf{F}$. So $p \to q \not\models q \to p$, and by soundness there is no proof.

5.3 Completeness

We follow the three-step proof from Huth and Ryan for finite premise sets, then outline the general (maximal consistent set) proof.

Step 1: reduce to theorems

If $\varphi_1, \ldots, \varphi_n \models \psi$, then

$$\models \varphi_1 \to (\varphi_2 \to (\ldots(\varphi_n \to \psi)\ldots)).$$

(If a valuation makes this false, it makes all φ_i true and ψ false.)

Step 2: every tautology is a theorem

Lemma 5.3 (Kalmár)**.** *Let φ have atoms $p_1, \ldots, p_n$, and let t be a valuation. Define $\hat{p}_i = p_i$ if $t(p_i) = \mathsf{T}$ and $\hat{p}_i = \neg p_i$ otherwise. Then*

$$\hat{p}_1, \ldots, \hat{p}_n \vdash \varphi \text{ if } \varphi^t = \mathsf{T}, \qquad \hat{p}_1, \ldots, \hat{p}_n \vdash \neg\varphi \text{ if } \varphi^t = \mathsf{F}.$$

Proof sketch. Structural induction on φ. Base: $\varphi = p_i$, and $\hat{p}_i$ is a premise. Step $\neg\varphi_1$: if $\varphi_1^t = \mathsf{T}$ then $(\neg\varphi_1)^t = \mathsf{F}$ and we need $\neg\neg\varphi_1$, obtained from φ_1 by $\neg\neg i$; otherwise IH gives $\neg\varphi_1$ directly. Step $\varphi_1 \to \varphi_2$: three cases. If $\varphi_2^t = \mathsf{T}$, derive φ_2 by IH and then $\varphi_1 \to \varphi_2$ by $\to i$ (with a box that just copies φ_2). If $\varphi_1^t = \mathsf{F}$, derive $\neg\varphi_1$, then inside a box assume φ_1, get $\bot$, then φ_2 by $\bot e$. If $\varphi_1^t = \mathsf{T}, \varphi_2^t = \mathsf{F}$, derive φ_1 and $\neg\varphi_2$ and then $\neg(\varphi_1 \to \varphi_2)$ (assume the implication, apply $\to e$, get $\bot$). The other connectives are similar. □

Now suppose $\models \eta$ with atoms $p_1, \ldots, p_n$. By the lemma, for each of the 2^n valuations, $\hat{p}_1, \ldots, \hat{p}_n \vdash \eta$. Use LEM on p_1: $p_1 \vee \neg p_1$, and apply $\vee e$. In each case, use LEM on p_2, and so on. At the leaves of this nested case analysis all of $\hat{p}_1, \ldots, \hat{p}_n$ are available, and we plug in the proof from the lemma. Altogether $\vdash \eta$.

Step 3: put the premises back

From $\vdash \varphi_1 \to (\cdots \to \psi)$, add premises $\varphi_1, \ldots, \varphi_n$ and use $\to e$ n times to obtain $\varphi_1, \ldots, \varphi_n \vdash \psi$. □

5.3.1 The general completeness theorem

For possibly infinite Σ we use consistency.

Definition 5.4. Σ is *consistent* if there is no φ with $\Sigma \vdash \varphi$ and $\Sigma \vdash \neg\varphi$ (equivalently, $\Sigma \nvdash \bot$). Σ is *maximal consistent* if it is consistent and for every formula φ, either $\varphi \in \Sigma$ or $\neg\varphi \in \Sigma$.

Lemma 5.5. *$\Sigma \nvdash \varphi$ iff $\Sigma \cup \{\neg\varphi\}$ is consistent.*

Theorem 5.6 (Lindenbaum's lemma)**.** *Every consistent set extends to a maximal consistent set.*

Sketch. Enumerate all formulas $\varphi_1, \varphi_2, \ldots$ (possible because $\mathcal{P}$ is countable). Let $\Sigma_0 = \Sigma$ and $\Sigma_{k+1} = \Sigma_k \cup \{\varphi_{k+1}\}$ if that is consistent, else $\Sigma_k \cup \{\neg\varphi_{k+1}\}$. One shows each Σ_k is consistent, and so is $\Sigma^* = \bigcup_k \Sigma_k$ because proofs are finite. □

Theorem 5.7. *Every consistent set is satisfiable.*

Sketch. Extend to a maximal consistent Σ^* and define $t(p) = \mathsf{T}$ iff $p \in \Sigma^*$. By structural induction, $\varphi^t = \mathsf{T}$ iff $\varphi \in \Sigma^*$ for every formula φ (e.g. $(\neg\varphi) \in \Sigma^*$ iff $\varphi \notin \Sigma^*$ by maximality and consistency). So t satisfies $\Sigma \subseteq \Sigma^*$. □

Corollary 5.8 (Completeness)**.** *If $\Sigma \models \varphi$ then $\Sigma \vdash \varphi$.*

Proof. If $\Sigma \nvdash \varphi$, then $\Sigma \cup \{\neg\varphi\}$ is consistent, hence satisfiable; a valuation satisfying it shows $\Sigma \not\models \varphi$. □

5.4 Compactness

Theorem 5.9 (Compactness)**.** *A set Σ of formulas is satisfiable iff every finite subset of Σ is satisfiable. Equivalently, $\Sigma \models \varphi$ iff $\Sigma_0 \models \varphi$ for some finite $\Sigma_0 \subseteq \Sigma$.*

Proof. If every finite subset is satisfiable, then (by soundness) every finite subset is consistent. Since a derivation of $\bot$ uses only finitely many premises, Σ is consistent, and by the theorem above, satisfiable. □

Compactness is a purely semantic fact proved here through the proof system: a nice illustration of why soundness and completeness are useful together.

5.5 Decidability

For finite Σ, $\Sigma \models \varphi$ is *decidable*: check all 2^n rows of the truth table. By completeness, provability $\Sigma \vdash \varphi$ is therefore decidable too. A polynomial-time algorithm for tautology checking exists iff P = NP, which is open (SAT is NP-complete; TAUT is co-NP-complete). We will see in Chapter 11 that the analogous problem for predicate logic is undecidable.

Summary

- Soundness ($\vdash \Rightarrow \models$): induction on proof lines, checking each rule preserves truth relative to the premises and open assumptions.
- Use soundness plus a counterexample valuation to show a sequent has no proof.
- Completeness ($\models \Rightarrow \vdash$): finite case via Kalmár's lemma and LEM case splits; general case via consistent $\Rightarrow$ maximal consistent $\Rightarrow$ satisfiable.
- Compactness: satisfiable iff every finite subset is satisfiable.
- Propositional validity is decidable (by truth tables), though possibly exponentially expensive.

Chapter 6

Predicate Logic: Syntax

6.1 Why propositional logic is not enough

Consider the argument:

> Every student in CS 245 is clever. Alice is a student in CS 245. Therefore Alice is clever.

In propositional logic these are three unrelated atoms p, q, r, and $\{p, q\} \not\models r$. The validity depends on the internal structure of the sentences: individuals (Alice), properties ("is clever"), and quantifiers ("every"). First-order predicate logic adds exactly these:

$$\forall x\,(S(x) \to C(x)),\ S(a)\ \models\ C(a).$$

6.2 The language

A first-order language $\mathcal{L}$ consists of logical symbols, common to all languages, and non-logical symbols, which depend on the application.

Logical symbols.

- connectives $\neg, \wedge, \vee, \to, \leftrightarrow$ and parentheses, comma;
- quantifiers $\forall$ (for all) and $\exists$ (there exists);
- an infinite supply of *variables* $x, y, z, u, v, x_1, \ldots$;
- (optionally) the equality symbol $=$.

Non-logical symbols (the signature).

- *constant symbols* $a, b, c, \ldots$ (names for individuals);
- *function symbols* $f, g, h, \ldots$, each with an arity $n \geq 1$;
- *predicate (relation) symbols* $P, Q, R, \ldots$, each with an arity $n \geq 0$. A 0-ary predicate is a propositional atom.

(Constants can be viewed as 0-ary function symbols.)

Example 6.1 (Language of arithmetic)**.** Constant 0; unary function s (successor); binary functions $+$ and $\cdot$ (written infix); binary predicate $<$; and equality.

6.3 Terms

Terms denote individuals.

Definition 6.2 (Term)**.** The set of terms is the smallest set such that

1. every variable and every constant symbol is a term;
2. if f is an n-ary function symbol and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term.

A term with no variables is a *closed* (ground) term.

Example 6.3. In arithmetic, 0, x, $s(s(0))$, $(x + s(0)) \cdot y$ are terms. $s(0) < x$ is *not* a term (it is a formula), and neither is s alone.

6.4 Formulas

Definition 6.4 (Atomic formula)**.** If P is an n-ary predicate symbol and $t_1, \dots, t_n$ are terms, then $P(t_1, \dots, t_n)$ is an atomic formula. If $=$ is present, $t_1 = t_2$ is also an atomic formula.

Definition 6.5 (Formula)**.** The set of formulas of $\mathcal{L}$ is the smallest set such that

1. every atomic formula is a formula;
2. if φ, ψ are formulas then so are $(\neg\varphi)$ and $(\varphi \star \psi)$ for $\star \in \{\wedge, \vee, \to, \leftrightarrow\}$;
3. if φ is a formula and x a variable, then $(\forall x\, \varphi)$ and $(\exists x\, \varphi)$ are formulas.

Quantifiers bind as tightly as $\neg$: $\forall x\, P(x) \to Q(x)$ means $(\forall x\, P(x)) \to Q(x)$. Use parentheses when the scope should extend further.

As in the propositional case, unique readability holds and we can draw parse trees: quantifier nodes $\forall x$ and $\exists x$ have one child, predicate nodes have their argument terms as children.

Terms vs. formulas

Terms (built from variables, constants, functions) denote *objects*; formulas (built from predicates, connectives, quantifiers) denote *truth values*. You cannot apply a function to a formula or a connective to a term: $f(P(x))$ and $\neg x$ are not well-formed.

6.5 Translating English

- "All As are Bs": $\forall x\, (A(x) \to B(x))$ $\quad$ (*not* $\wedge$).
- "Some A is a B": $\exists x\, (A(x) \wedge B(x))$ $\quad$ (*not* $\to$).
- "No A is a B": $\forall x\, (A(x) \to \neg B(x))$, or $\neg\exists x\, (A(x) \wedge B(x))$.
- "Not all As are Bs": $\exists x\, (A(x) \wedge \neg B(x))$.
- "There is exactly one x with P": $\exists x\, (P(x) \wedge \forall y\, (P(y) \to y = x))$.
- "There are at least two": $\exists x\, \exists y\, (\neg(x = y) \wedge P(x) \wedge P(y))$.

Pitfall: $\wedge$ vs. $\to$ with quantifiers

$\forall x\,(S(x) \wedge C(x))$ says *everything* is a clever student. $\exists x\,(S(x) \to C(x))$ is true as soon as there is any non-student, so it says almost nothing. Pair $\forall$ with $\to$ and $\exists$ with $\wedge$.

Example 6.6 (Order of quantifiers)**.** With $L(x, y)$ meaning "x loves y":

- $\forall x\,\exists y\,L(x, y)$: everybody loves somebody (possibly different people);
- $\exists y\,\forall x\,L(x, y)$: there is one person whom everybody loves.

The second implies the first but not conversely.

Example 6.7. "Every natural number has a larger prime": $\forall x\,\exists y\,(x < y \wedge \mathit{Prime}(y))$, where $\mathit{Prime}(y)$ could itself be defined as $s(0) < y \wedge \forall u\,\forall v\,(u \cdot v = y \to (u = s(0) \vee v = s(0)))$.

6.6 Free and bound variables

Definition 6.8 (Scope, free and bound occurrences)**.** In $\forall x\,\varphi$ or $\exists x\,\varphi$, the formula φ is the *scope* of the quantifier. An occurrence of x is *bound* if it lies within the scope of a quantifier $\forall x$ or $\exists x$ (or is the x right after the quantifier); otherwise it is *free*.

The set of free variables is defined recursively:

$$\begin{aligned} FV(P(t_1, \dots, t_n)) &= \mathrm{vars}(t_1) \cup \dots \cup \mathrm{vars}(t_n), \\ FV(\neg\varphi) = FV(\varphi), &\qquad FV(\varphi \star \psi) = FV(\varphi) \cup FV(\psi), \\ FV(\forall x\,\varphi) = FV(\exists x\,\varphi) &= FV(\varphi) \setminus \{x\}. \end{aligned}$$

Definition 6.9 (Sentence)**.** A formula with no free variables is a *sentence* (closed formula). Only sentences have a truth value once an interpretation is fixed; a formula with free variables also needs values for those variables.

Example 6.10. In $\forall x\,(P(x, y) \to \exists y\,Q(y, x)) \wedge R(x)$:

- the x in $P(x, y)$ and in $Q(y, x)$ are bound by $\forall x$;
- the y in $P(x, y)$ is *free*; the y in $Q(y, x)$ is bound by $\exists y$;
- the x in $R(x)$ is free (outside the scope of $\forall x$).

So $FV = \{x, y\}$. The same variable can occur both free and bound in one formula.

Bound variables are "dummy" names: $\forall x\,P(x)$ and $\forall y\,P(y)$ say the same thing (they are *alpha-equivalent*), just as $\sum_i a_i = \sum_j a_j$.

6.7 Substitution

Definition 6.11 (Substitution)**.** $\varphi[t/x]$ denotes the result of replacing every *free* occurrence of x in φ by the term t. Formally, on terms $x[t/x] = t$, $y[t/x] = y$ for $y \neq x$, and $f(t_1, \dots)[t/x] = f(t_1[t/x], \dots)$; on formulas it passes through connectives, and

$$(\forall y\,\varphi)[t/x] = \begin{cases} \forall y\,\varphi & \text{if } y = x \text{ (x is not free)}, \\ \forall y\,(\varphi[t/x]) & \text{if } y \neq x, \end{cases}$$

and similarly for $\exists$.

Definition 6.12 (Free for)**.** A term t is *free for* x in φ if no free occurrence of x in φ lies within the scope of a quantifier $\forall y$ or $\exists y$ for some variable y occurring in t. In words: substituting t does not cause any variable of t to be *captured*.

Example 6.13 (Variable capture)**.** Let $\varphi = \exists y\,(x < y)$ ("something is bigger than x"; true in $\mathbb{N}$ for every x). Then

- $\varphi[s(z)/x] = \exists y\,(s(z) < y)$: fine, $s(z)$ is free for x.
- $\varphi[y/x] = \exists y\,(y < y)$: wrong! The y got captured, and the meaning changed to "something is bigger than itself" (false). y is not free for x.

The fix is to rename the bound variable first: $\exists y\,(x < y) \equiv \exists w\,(x < w)$, so $\varphi[y/x]$ should be taken as $\exists w\,(y < w)$.

Substitution rules of thumb

- Only *free* occurrences are replaced.
- Before substituting t for x, rename any bound variables of φ that occur in t and whose quantifiers have free x in their scope.
- Every rule in Chapter 8 that substitutes ($\forall e$, $\exists i$) requires t to be free for x.

Summary

- A first-order language has constants, function symbols and predicate symbols, plus variables, connectives, quantifiers and possibly $=$.
- Terms denote objects; atomic formulas apply predicates to terms; formulas close under connectives and quantifiers.
- Translate "all A are B" with $\forall$ and $\rightarrow$; "some A is B" with $\exists$ and $\wedge$; quantifier order matters.
- Free vs. bound occurrences; sentences have no free variables.
- $\varphi[t/x]$ replaces free x by t; it is only safe when t is free for x (no capture); rename bound variables otherwise.

Chapter 7

Predicate Logic: Semantics

In propositional logic, a valuation (one truth value per atom) is all the meaning we need. In predicate logic we must say what the objects are and what each symbol denotes.

7.1 Interpretations

Definition 7.1 (Interpretation / structure / model)**.** An *interpretation* I for a language $\mathcal{L}$ consists of

1. a non-empty set D, the *domain* (universe of discourse);
2. for each constant symbol c, an element $c^I \in D$;
3. for each n-ary function symbol f, a function $f^I : D^n \to D$;
4. for each n-ary predicate symbol P, a relation $P^I \subseteq D^n$ (for $n = 0$, a truth value).

The equality symbol, if present, is always interpreted as true identity on D.

Domains are non-empty and functions are total

The domain must be non-empty, and each f^I must be defined on *all* of D^n with values in D. E.g. subtraction is not a valid interpretation of a binary function symbol on domain $\mathbb{N}$ (since $2 - 5 \notin \mathbb{N}$) unless modified (truncated subtraction).

7.2 Environments and evaluation

Formulas with free variables need values for them.

Definition 7.2 (Environment)**.** An *environment* (variable assignment) for I is a function E from variables to D. For $d \in D$, $E[x \mapsto d]$ is the environment equal to E except it maps x to d.

Definition 7.3 (Value of a term)**.**

$$x^{(I,E)} = E(x), \qquad c^{(I,E)} = c^I, \qquad f(t_1, \ldots, t_n)^{(I,E)} = f^I(t_1^{(I,E)}, \ldots, t_n^{(I,E)}).$$

Definition 7.4 (Satisfaction)**.** We define $I \models_E \varphi$ (I *satisfies* φ *under* E) recursively:

- $I \models_E P(t_1, \ldots, t_n)$ iff $(t_1^{(I,E)}, \ldots, t_n^{(I,E)}) \in P^I$;
- $I \models_E t_1 = t_2$ iff $t_1^{(I,E)} = t_2^{(I,E)}$;
- connectives as in propositional logic, e.g. $I \models_E \neg\varphi$ iff $I \not\models_E \varphi$, and $I \models_E \varphi \to \psi$ iff $I \not\models_E \varphi$ or $I \models_E \psi$;
- $I \models_E \forall x\, \varphi$ iff $I \models_{E[x\mapsto d]} \varphi$ for *every* $d \in D$;
- $I \models_E \exists x\, \varphi$ iff $I \models_{E[x\mapsto d]} \varphi$ for *some* $d \in D$.

Lemma 7.5. *Whether $I \models_E \varphi$ depends only on E's values on $FV(\varphi)$. In particular, for a sentence φ the environment is irrelevant and we write $I \models \varphi$ ("φ is true in I", "I is a model of φ").*

Example 7.6. Language: binary predicate R, constant c. Let $\varphi = \forall x\, \exists y\, R(x, y)$ and $\psi = \exists y\, \forall x\, R(x, y)$.

- I_1: $D = \mathbb{N}$, $R^{I_1} = <$. Then $I_1 \models \varphi$ (for each n take $n + 1$) but $I_1 \not\models \psi$ (no largest natural number).
- I_2: $D = \mathbb{N}$, $R^{I_2} = \geq$. Then $I_2 \models \psi$ (take $y = 0$, since $x \geq 0$ for all x), so also $I_2 \models \varphi$.
- I_3: $D = \{1, 2\}$, $R^{I_3} = \{(1, 2)\}$. Then $I_3 \not\models \varphi$ since 2 is not related to anything.

Example 7.7 (Evaluating with an environment)**.** I: $D = \mathbb{Z}$, $f^I(a, b) = a + b$, $c^I = 0$, $P^I = \{(a, b) : a < b\}$. Let $\varphi = \exists y\, P(f(x, y), c)$ and $E(x) = 5$. We need some $d \in \mathbb{Z}$ with $5 + d < 0$; $d = -6$ works, so $I \models_E \varphi$. In fact $I \models_E \varphi$ for every E, so $I \models \forall x\, \varphi$.

7.3 Validity, satisfiability, consequence

Definition 7.8.

- φ is *satisfiable* if $I \models_E \varphi$ for some I and E.
- φ is *valid* (logically true), written $\models \varphi$, if $I \models_E \varphi$ for every I and E.
- A set Σ is satisfiable if some single (I, E) satisfies every formula in Σ.
- $\Sigma \models \varphi$ (φ *is a logical consequence of* Σ) if every (I, E) that satisfies all of Σ also satisfies φ.
- $\varphi \equiv \psi$ if $\varphi \models \psi$ and $\psi \models \varphi$.

The same facts as before hold: $\Sigma \models \varphi$ iff $\Sigma \cup \{\neg\varphi\}$ is unsatisfiable, and $\Sigma \cup \{\varphi\} \models \psi$ iff $\Sigma \models \varphi \to \psi$ (for sentences φ).

How to prove/disprove semantic claims

- To show $\Sigma \not\models \varphi$ (or that φ is not valid): give a *specific* interpretation (domain + meaning of every symbol) making all of Σ true and φ false. Small finite domains are often easiest.
- To show $\Sigma \models \varphi$: take an *arbitrary* interpretation I and environment E satisfying Σ, and reason from the definition of satisfaction that $I \models_E \varphi$.

Example 7.9 (Proving a consequence)**.** Show $\forall x\,(P(x) \to Q(x)),\ \forall x\,P(x) \models \forall x\,Q(x)$.

Let (I, E) satisfy both premises and let $d \in D$ be arbitrary. By the second premise, $d \in P^I$. By the first, with $x \mapsto d$, if $d \in P^I$ then $d \in Q^I$. So $d \in Q^I$. Since d was arbitrary, $I \models_E \forall x\,Q(x)$.

Example 7.10 (Disproving a consequence)**.** Show $\forall x\,(P(x) \lor Q(x)) \not\models \forall x\,P(x) \lor \forall x\,Q(x)$.

Take $D = \{1, 2\}$, $P^I = \{1\}$, $Q^I = \{2\}$. Every element is in P^I or Q^I, so the premise holds; but neither P^I nor Q^I is all of D, so the conclusion fails.

Example 7.11. $\exists x\,\forall y\,R(x, y) \models \forall y\,\exists x\,R(x, y)$ is valid: if d_0 is related to everything, then for each e we can pick $x = d_0$. The converse fails: interpret R as equality on a two-element domain. Everything equals something, but nothing equals everything.

7.4 Important equivalences

Quantifier laws

Negation (De Morgan)	$\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi, \quad \neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$
Distribution	$\forall x\,(\varphi \land \psi) \equiv \forall x\,\varphi \land \forall x\,\psi$
	$\exists x\,(\varphi \lor \psi) \equiv \exists x\,\varphi \lor \exists x\,\psi$
Commutation	$\forall x\,\forall y\,\varphi \equiv \forall y\,\forall x\,\varphi, \quad \exists x\,\exists y\,\varphi \equiv \exists y\,\exists x\,\varphi$
If $x \notin FV(\psi)$:	
	$\forall x\,\varphi \land \psi \equiv \forall x\,(\varphi \land \psi), \quad \exists x\,\varphi \lor \psi \equiv \exists x\,(\varphi \lor \psi)$
	$\forall x\,\varphi \lor \psi \equiv \forall x\,(\varphi \lor \psi), \quad \exists x\,\varphi \land \psi \equiv \exists x\,(\varphi \land \psi)$
	$\psi \to \forall x\,\varphi \equiv \forall x\,(\psi \to \varphi), \quad \psi \to \exists x\,\varphi \equiv \exists x\,(\psi \to \varphi)$
	$(\forall x\,\varphi) \to \psi \equiv \exists x\,(\varphi \to \psi), \quad (\exists x\,\varphi) \to \psi \equiv \forall x\,(\varphi \to \psi)$

Note the flip in the last line: a quantifier in the antecedent of $\to$ changes type when pulled out. Also note the *non*-equivalences: $\forall x\,(\varphi \lor \psi) \not\equiv \forall x\,\varphi \lor \forall x\,\psi$ and $\exists x\,(\varphi \land \psi) \not\equiv \exists x\,\varphi \land \exists x\,\psi$ (only $\models$ in one direction).

Using these laws (and renaming bound variables), every formula can be converted to *prenex normal form*: a string of quantifiers followed by a quantifier-free matrix, e.g. $\forall x\,\exists y\,\forall z\,(\dots)$. Prenex form plus *Skolemisation* (replacing $\exists y$ inside $\forall x$ by a new function $f(x)$) and CNF conversion is how resolution is extended to first-order logic, together with *unification* of terms.

7.5 Theories and models

A set of sentences Σ often serves as axioms describing a class of structures.

Example 7.12 (Partial orders)**.** Language: binary predicate $\leq$. Axioms:

$$\forall x\,(x \leq x), \quad \forall x\,\forall y\,(x \leq y \land y \leq x \to x = y),$$
$$\forall x\,\forall y\,\forall z\,(x \leq y \land y \leq z \to x \leq z).$$

Models include $(\mathbb{N}, \leq)$, $(\mathcal{P}(S), \subseteq)$ and $(\mathbb{Z}^+, |)$. Adding $\forall x\,\forall y\,(x \leq y \lor y \leq x)$ excludes $\subseteq$ and $|$.

Example 7.13 (Expressing size)**.** $\exists x\,\exists y\,\neg(x = y)$ is true exactly in interpretations with at least two elements. $\forall x\,\forall y\,(x = y)$ holds exactly when $|D| = 1$. With these one can express "at least n" and "at most n" for each fixed n, but (by compactness) no set of sentences is true in exactly the finite interpretations.

Summary

- An interpretation gives a non-empty domain and meanings for constants, functions (total) and predicates; an environment gives values to variables.
- $\forall x\, \varphi$ is true iff φ holds for every value of x; $\exists$ iff for some value.
- Valid: true in every (I, E); satisfiable: true in some; $\Sigma \models \varphi$: every model of Σ is a model of φ.
- Disprove with a concrete counter-interpretation; prove by reasoning about an arbitrary one.
- Quantifier laws: De Morgan, distribution of $\forall$ over $\wedge$ and $\exists$ over $\vee$, pulling quantifiers past formulas without the variable free.

Chapter 8

Natural Deduction for Predicate Logic

Natural deduction for predicate logic keeps all the propositional rules (now applied to first-order formulas) and adds introduction and elimination rules for $\forall$, $\exists$ and $=$.

8.1 Rules for the quantifiers

8.1.1 $\forall$ elimination

$$\frac{\forall x\, \varphi}{\varphi[t/x]}\ \forall e \qquad (t \text{ any term free for } x \text{ in } \varphi).$$

If something holds for everything, it holds for any particular t.

8.1.2 $\forall$ introduction

$$\frac{\boxed{\begin{array}{c} x_0 \text{ (fresh)} \\ \vdots \\ \varphi[x_0/x] \end{array}}}{\forall x\, \varphi}\ \forall i$$

Open a box with a *fresh* variable x_0 that appears nowhere outside the box (not in premises, not in open assumptions, not in the conclusion). If we can prove $\varphi[x_0/x]$ about this completely arbitrary x_0, the proof works for every value, so $\forall x\, \varphi$ follows. This is the formal version of "let x_0 be arbitrary".

8.1.3 $\exists$ introduction

$$\frac{\varphi[t/x]}{\exists x\, \varphi}\ \exists i \qquad (t \text{ free for } x \text{ in } \varphi).$$

A concrete witness t gives existence.

8.1.4 $\exists$ elimination

$$\frac{\exists x\, \varphi \qquad \boxed{\begin{array}{cc} x_0 & \varphi[x_0/x] \text{ (assumption)} \\ & \vdots \\ & \chi \end{array}}}{\chi}\ \exists e$$

Knowing that *some* object satisfies φ, give it a fresh name x_0, assume $\varphi[x_0/x]$, and derive χ. Provided x_0 does not occur in χ (or anywhere outside the box), χ follows. This is like $\vee e$ over all possible witnesses.

Freshness conditions

- In $\forall i$ and $\exists e$, the variable x_0 must be new: it must not occur in any premise, any open assumption, or the formula concluded after the box.
- In $\exists e$ the conclusion χ must not mention x_0: we do not know *which* object the witness is.
- In $\forall e$ and $\exists i$, t must be free for x (no variable capture).

Violating these is the most common source of "proofs" of false statements, e.g. deriving $\forall x\, P(x)$ from $\exists x\, P(x)$ by using the witness as if it were arbitrary.

8.2 Rules for equality

$$\frac{}{t = t}\ {=}i \qquad\qquad \frac{t_1 = t_2 \qquad \varphi[t_1/x]}{\varphi[t_2/x]}\ {=}e$$

(t_1, t_2 free for x in φ.) $= e$ is "substitution of equals for equals".

Example 8.1 (Symmetry of $=$: $t_1 = t_2 \vdash t_2 = t_1$)**.**

1	$t_1 = t_2$	premise	
2	$t_1 = t_1$	$= i$	
3	$t_2 = t_1$	$= e$ 1, 2	(with φ being $x = t_1$)

Transitivity ($t_1 = t_2,\ t_2 = t_3 \vdash t_1 = t_3$) is a single $= e$ with φ being $t_1 = x$.

8.3 Example proofs

Example 8.2 ($\forall x\,(P(x) \to Q(x)),\ \forall x\, P(x) \vdash \forall x\, Q(x)$)**.**

1	$\forall x\,(P(x) \to Q(x))$	premise
2	$\forall x\, P(x)$	premise
3	$\quad P(x_0) \to Q(x_0)$	x_0 fresh; $\forall e$ 1
4	$\quad P(x_0)$	$\forall e$ 2
5	$\quad Q(x_0)$	$\to e$ 4, 3
6	$\forall x\, Q(x)$	$\forall i$ 3–5

Example 8.3 ($\forall x\,(P(x) \to Q(x)),\ \exists x\, P(x) \vdash \exists x\, Q(x)$)**.**

1	$\forall x\,(P(x) \to Q(x))$	premise
2	$\exists x\, P(x)$	premise
3	$\quad P(x_0)$	assumption, x_0 fresh
4	$\quad P(x_0) \to Q(x_0)$	$\forall e$ 1
5	$\quad Q(x_0)$	$\to e$ 3, 4
6	$\quad \exists x\, Q(x)$	$\exists i$ 5
7	$\exists x\, Q(x)$	$\exists e$ 2, 3–6

Note line 6: we must reach a formula without x_0 *inside* the box before closing it.

Example 8.4 $(\neg\exists x\, P(x) \vdash \forall x\, \neg P(x))$**.**

1	$\neg\exists x\, P(x)$	premise
2	$\quad P(x_0)$	x_0 fresh; assumption
3	$\quad \exists x\, P(x)$	$\exists i$ 2
4	$\quad \bot$	$\neg e$ 3, 1
5	$\neg P(x_0)$	$\neg i$ 2–4
6	$\forall x\, \neg P(x)$	$\forall i$ 2–5

The outer box (lines 2–5) is the $\forall i$ box for x_0; the inner one (2–4) is the $\neg i$ box.

Example 8.5 $(\neg\forall x\, P(x) \vdash \exists x\, \neg P(x))$**.** This direction needs classical reasoning (PBC).

1	$\neg\forall x\, P(x)$	premise
2	$\quad \neg\exists x\, \neg P(x)$	assumption
3	$\quad\quad \neg P(x_0)$	x_0 fresh; assumption
4	$\quad\quad \exists x\, \neg P(x)$	$\exists i$ 3
5	$\quad\quad \bot$	$\neg e$ 4, 2
6	$\quad P(x_0)$	PBC 3–5
7	$\quad \forall x\, P(x)$	$\forall i$ 3–6
8	$\quad \bot$	$\neg e$ 7, 1
9	$\exists x\, \neg P(x)$	PBC 2–8

Example 8.6 $(\exists x\, \forall y\, R(x,y) \vdash \forall y\, \exists x\, R(x,y))$**.**

1	$\exists x\, \forall y\, R(x,y)$	premise
2	$\quad \forall y\, R(x_0,y)$	assumption, x_0 fresh
3	$\quad\quad R(x_0,y_0)$	y_0 fresh; $\forall e$ 2
4	$\quad\quad \exists x\, R(x,y_0)$	$\exists i$ 3
5	$\quad \forall y\, \exists x\, R(x,y)$	$\forall i$ 3–4
6	$\forall y\, \exists x\, R(x,y)$	$\exists e$ 1, 2–5

Trying to prove the converse fails exactly at a freshness condition: from $\forall y\, \exists x\, R(x,y)$ we would get a witness x_0 depending on y_0, and could not then generalise over y_0 while concluding something about x_0.

Example 8.7 $(\exists x\, (P(x) \wedge Q(x)) \vdash \exists x\, P(x) \wedge \exists x\, Q(x))$**.**

1	$\exists x\, (P(x) \wedge Q(x))$	premise
2	$\quad P(x_0) \wedge Q(x_0)$	assumption, x_0 fresh
3	$\quad P(x_0)$	$\wedge e_1$ 2
4	$\quad \exists x\, P(x)$	$\exists i$ 3
5	$\quad Q(x_0)$	$\wedge e_2$ 2
6	$\quad \exists x\, Q(x)$	$\exists i$ 5
7	$\quad \exists x\, P(x) \wedge \exists x\, Q(x)$	$\wedge i$ 4, 6
8	$\exists x\, P(x) \wedge \exists x\, Q(x)$	$\exists e$ 1, 2–7

Strategy

- Goal $\forall x\,\varphi$: open a $\forall i$ box with fresh x_0; aim for $\varphi[x_0/x]$.
- Goal $\exists x\,\varphi$: find a term t and prove $\varphi[t/x]$; if no witness is apparent, try PBC.
- Premise $\exists x\,\varphi$: open an $\exists e$ box *early*, so the witness x_0 is available, and close it once the goal (without x_0) is reached.
- Premise $\forall x\,\varphi$: instantiate with $\forall e$ at whatever terms you need (often the fresh variables of your boxes).

8.4 Soundness and completeness

Theorem 8.8 (Soundness)**.** *If $\Sigma \vdash \varphi$ in natural deduction for predicate logic, then $\Sigma \models \varphi$.*

The proof extends the propositional one: induction on proof lines, checking each new rule preserves truth in every (I, E). For $\forall i$, freshness of x_0 means the box's argument works under $E[x_0 \mapsto d]$ for every $d \in D$; for $\exists e$, freshness lets us pick $E[x_0 \mapsto d]$ with d a witness, and since χ does not mention x_0 its truth is unaffected.

Theorem 8.9 (Completeness, Gödel 1929)**.** *If $\Sigma \models \varphi$ then $\Sigma \vdash \varphi$.*

Idea (Henkin). Show every consistent set of sentences has a model. Extend Σ to a maximal consistent set with *witnesses*: for every sentence $\exists x\,\varphi$ add a new constant c and the sentence $\exists x\,\varphi \to \varphi[c/x]$. Then build a model whose domain is the set of closed terms (modulo provable equality), interpreting each predicate as the set of tuples for which the atomic sentence is in the maximal set. □

Corollary 8.10 (Compactness)**.** *A set of first-order sentences has a model iff every finite subset has a model.*

Consequences of compactness: if a set of sentences has arbitrarily large finite models then it has an infinite model (so "the domain is finite" is not first-order expressible); and there are non-standard models of arithmetic containing "infinite" numbers (see Chapter 9). The Löwenheim–Skolem theorem adds that any theory with an infinite model has a countable one.

Semi-decidability

Completeness implies that the set of valid first-order formulas is *semi-decidable*: enumerate all proofs and check each; if φ is valid, a proof will eventually be found. But if φ is not valid the search may run forever. Church and Turing showed (Chapter 11) that no algorithm decides validity in general.

Summary

- $\forall e$: instantiate at any term free for x. $\exists i$: give a witness.
- $\forall i$: box with fresh x_0, prove $\varphi[x_0/x]$. $\exists e$: box with fresh x_0 assuming $\varphi[x_0/x]$, derive χ not mentioning x_0.

- $= i$: $t = t$; $= e$: replace equals by equals; gives symmetry and transitivity.
- $\neg\forall \leftrightarrow \exists\neg$ needs PBC in one direction.
- Soundness and completeness hold (Gödel); hence compactness, and validity is semi-decidable but not decidable.

Chapter 9

Peano Arithmetic and Formal Number Theory

Having a sound and complete proof system for first-order logic, we can now pick a set of axioms and study what follows. The most important example is arithmetic on the natural numbers. This chapter also revisits induction, the main proof technique for reasoning about programs and data structures.

9.1 The language of arithmetic

The language $\mathcal{L}_A$ has

- a constant symbol 0;
- a unary function symbol s (successor; think $s(x) = x + 1$);
- binary function symbols $+$ and $\cdot$ (written infix);
- equality $=$ (and sometimes a binary predicate $<$, which can also be defined).

The *standard interpretation* $\mathcal{N}$ has domain $\mathbb{N} = \{0, 1, 2, \dots\}$ with the usual meanings. The closed term $s(s(s(0)))$ denotes 3; we abbreviate $s^n(0)$ by $\overline{n}$ (a *numeral*).

9.2 The Peano axioms

Definition 9.1 (Peano arithmetic, PA)**.**

$$
\begin{aligned}
&\text{(PA1)} && \forall x\, \neg(s(x) = 0)\\
&\text{(PA2)} && \forall x\, \forall y\, (s(x) = s(y) \to x = y)\\
&\text{(PA3)} && \forall x\, (x + 0 = x)\\
&\text{(PA4)} && \forall x\, \forall y\, (x + s(y) = s(x + y))\\
&\text{(PA5)} && \forall x\, (x \cdot 0 = 0)\\
&\text{(PA6)} && \forall x\, \forall y\, (x \cdot s(y) = x \cdot y + x)\\
&\text{(PA7, induction scheme)} && (\varphi[0/x] \land \forall x\, (\varphi \to \varphi[s(x)/x])) \to \forall x\, \varphi
\end{aligned}
$$

for every formula φ (which may have other free variables, implicitly universally quantified).

- PA1 and PA2 say 0 is not a successor and s is injective, so $0, s(0), s(s(0)), \ldots$ are all distinct.
- PA3–PA6 are the recursive definitions of $+$ and $\cdot$ (recursion on the second argument).
- PA7 is not one axiom but infinitely many: one for each formula φ. Its job is to rule out elements that are not reachable from 0 by finitely many successor steps (as far as first-order logic can).

Order can be defined: $x \le y$ abbreviates $\exists z\,(x + z = y)$ and $x < y$ abbreviates $\exists z\,(x + s(z) = y)$.

9.3 Computing with the axioms

Example 9.2 ($\overline{2} + \overline{2} = \overline{4}$)**.** Informally, using PA4 twice and PA3 once:

$$\begin{aligned} s(s(0)) + s(s(0)) &= s(s(s(0)) + s(0)) && \text{PA4} \\ &= s(s(s(s(0)) + 0)) && \text{PA4} \\ &= s(s(s(s(0)))) && \text{PA3 and } = e. \end{aligned}$$

Every true equation between closed terms is provable this way: PA (in fact, the much weaker system without induction) proves all true closed atomic sentences.

9.4 Proofs by induction in PA

To prove $\forall x\,\varphi$ in PA, prove the base case $\varphi[0/x]$ and the inductive step $\forall x\,(\varphi \to \varphi[s(x)/x])$, then apply PA7 and $\to e$.

Example 9.3 ($\vdash_{PA} \forall x\,(0 + x = x)$)**.** Note this is *not* an axiom: PA3 says $x + 0 = x$, with 0 on the right. Let φ be $0 + x = x$.

1	$\forall x\,(x + 0 = x)$	PA3
2	$\forall x\,\forall y\,(x + s(y) = s(x + y))$	PA4
3	$0 + 0 = 0$	$\forall e$ 1
4	$\quad 0 + x_0 = x_0$	x_0 fresh; assumption
5	$\quad \forall y\,(0 + s(y) = s(0 + y))$	$\forall e$ 2
6	$\quad 0 + s(x_0) = s(0 + x_0)$	$\forall e$ 5
7	$\quad 0 + s(x_0) = s(x_0)$	$= e$ 4, 6
8	$0 + x_0 = x_0 \to 0 + s(x_0) = s(x_0)$	$\to i$ 4–7
9	$\forall x\,(0 + x = x \to 0 + s(x) = s(x))$	$\forall i$ 4–8
10	$0 + 0 = 0 \wedge \forall x\,(0 + x = x \to 0 + s(x) = s(x))$	$\wedge i$ 3, 9
11	$(0 + 0 = 0 \wedge \forall x\,(\ldots)) \to \forall x\,(0 + x = x)$	PA7
12	$\forall x\,(0 + x = x)$	$\to e$ 10, 11

In line 7, $= e$ uses φ' being $0 + s(x_0) = s(z)$: line 6 is $\varphi'[0 + x_0/z]$ and line 4 says $0 + x_0 = x_0$, giving $\varphi'[x_0/z]$.

Example 9.4 ($\forall x\,\neg(s(x) = x)$, informally)**.** Let $\varphi(x)$ be $\neg(s(x) = x)$. *Base*: $\neg(s(0) = 0)$ is an instance of PA1. *Step*: assume $\neg(s(x_0) = x_0)$. If $s(s(x_0)) = s(x_0)$, then by PA2, $s(x_0) = x_0$, contradicting the IH. So $\neg(s(s(x_0)) = s(x_0))$. By PA7, $\forall x\,\neg(s(x) = x)$.

Example 9.5 (Commutativity of $+$, sketch)**.** 1. Lemma A: $\forall x\,(0 + x = x)$ (above).

2. Lemma B: $\forall x\,\forall y\,(s(y)+x=s(y+x))$, by induction on x: base $s(y)+0=s(y)=s(y+0)$ by PA3; step $s(y)+s(x)=s(s(y)+x)=s(s(y+x))=s(y+s(x))$ by PA4, IH, PA4.

3. Theorem: $\forall x\,\forall y\,(x+y=y+x)$, by induction on y: base $x+0=x=0+x$ by PA3 and Lemma A; step $x+s(y)=s(x+y)=s(y+x)=s(y)+x$ by PA4, IH, Lemma B.

Choosing which variable to induct on, and which auxiliary lemmas to prove first, is the creative part.

9.5 Forms of induction

Equivalent induction principles on $\mathbb{N}$

Simple induction: $P(0)$ and $\forall n\,(P(n)\to P(n+1))$ imply $\forall n\,P(n)$.

Strong (complete) induction: if $\forall n\,((\forall k<n\;P(k))\to P(n))$ then $\forall n\,P(n)$. (No separate base case is needed, but the case $n=0$ must be handled with no hypotheses.)

Well-ordering: every non-empty subset of $\mathbb{N}$ has a least element. A proof by "minimal counterexample" uses this.

Strong induction follows from simple induction applied to $Q(n)=\forall k\le n\;P(k)$.

Structural induction (Chapter 2) generalises these to any inductively defined set: formulas, lists, trees. Induction on the length of a proof (used in the soundness proof) is strong induction on $\mathbb{N}$.

Example 9.6 (Strong induction)**.** Every integer $n\ge 2$ is a product of primes. If n is prime, done. Otherwise $n=ab$ with $2\le a,b<n$; by IH both a and b are products of primes, hence so is n.

Common pitfalls

- Forgetting the base case ("$n=n+1$ implies $n+1=n+2$" proves nothing).
- An inductive step that silently fails for small n (the classic "all horses are the same colour" argument breaks going from 1 to 2).
- Using the conclusion $\forall x\,\varphi$ inside the step: the IH is only φ for the particular x_0.
- Inducting on the wrong variable, or with a statement too weak to carry the step; strengthening the claim ("loading the induction") often helps.

9.6 Models of PA and the limits of first-order arithmetic

- The standard model $\mathcal{N}$ satisfies all axioms of PA, so (by soundness) PA is consistent if we accept that $\mathcal{N}$ exists.
- **Non-standard models.** Add a new constant c and sentences $\overline{n}<c$ for every n. Every finite subset is satisfied in $\mathcal{N}$ (interpret c as a large number), so by compactness the whole set has a model: a model of PA with an element larger than every numeral. So first-order PA cannot pin down $\mathbb{N}$ up to isomorphism. (Second-order induction, quantifying over all subsets, can.)

- **Incompleteness.** Gödel (1931) showed there are sentences true in $\mathcal{N}$ that PA cannot prove, and that this cannot be fixed by adding more (computably listable) axioms. See Chapter 11.
- By contrast, *Presburger arithmetic* (only 0, s, $+$, no $\cdot$) is complete and decidable.

The connection to computer science: the recursive equations PA3–PA6 are exactly how addition and multiplication would be programmed on unary numbers:

```
(define (add x y)            ; recursion on y, like PA3/PA4
  (cond [(zero? y) x]
        [else (add1 (add x (sub1 y)))]))
```

and an induction proof of a property of `add` follows the shape of its recursion.

Summary

- PA: 0 is not a successor; s is injective; recursive definitions of $+$ and $\cdot$; and the induction scheme (one axiom per formula).
- Facts like $0 + x = x$ and $x + y = y + x$ need induction; closed equations just need PA3–PA6.
- Simple induction, strong induction and well-ordering are equivalent on $\mathbb{N}$; structural induction generalises them.
- First-order PA has non-standard models (compactness) and is incomplete (Gödel).

Chapter 10

Program Verification

Testing can show the presence of bugs but not their absence. *Formal verification* proves that a program meets its specification for *all* inputs. We use Floyd–Hoare logic for a small imperative language, as in Huth and Ryan.

10.1 The programming language

Programs (commands) C are built from integer expressions E and Boolean expressions B:

$$C ::= x = E \mid C_1; C_2 \mid \texttt{if}\ (B)\ \{C_1\}\ \texttt{else}\ \{C_2\} \mid \texttt{while}\ (B)\ \{C\}$$

Expressions are side-effect free (they always terminate and do not change variables), and variables hold unbounded integers. Later we add arrays.

10.2 Hoare triples

Definition 10.1 (Hoare triple)**.** A *Hoare triple* $\{\,P\,\}\ C\ \{\,Q\,\}$ consists of a *precondition* P, a program C, and a *postcondition* Q, where P, Q are first-order formulas over the program variables (and arithmetic).

- $\{\,P\,\}\ C\ \{\,Q\,\}$ is satisfied under *partial correctness*, written $\models_{\text{par}} \{\,P\,\}\ C\ \{\,Q\,\}$, if for every state satisfying P, *if* C terminates, the final state satisfies Q.
- It is satisfied under *total correctness*, written $\models_{\text{tot}} \{\,P\,\}\ C\ \{\,Q\,\}$, if for every state satisfying P, C *is guaranteed to terminate* and the final state satisfies Q.

Total correctness = partial correctness + termination.

Example 10.2.

- $\{\,x = 5\,\}\ y = x + 1\ \{\,y = 6\,\}$ holds (partially and totally).
- $\{\,\top\,\}$ `while (true) { x = x }` $\{\,x = 42\,\}$ holds under partial correctness (vacuously: it never terminates) but not total correctness.
- $\{\,\top\,\}\ y = x \cdot x\ \{\,y > 0\,\}$ fails: $x = 0$ gives $y = 0$.

Logical variables

Specifications often need to refer to initial values. Use *logical variables* (which the program never assigns), e.g. $\{\, x = x_0 \wedge y = y_0 \,\}\ C_{\text{swap}}\ \{\, x = y_0 \wedge y = x_0 \,\}$. Also beware trivial "solutions": $\{\, x \geq 0 \,\}\ y = 1;\ x = 0\ \{\, y = x! \,\}$ is correct because the postcondition did not pin down x. Use logical variables to prevent this.

10.3 Proof rules for partial correctness

Assignment
$$\frac{}{\{\, Q[E/x] \,\}\ x = E\ \{\, Q \,\}}$$

Composition
$$\frac{\{\, P \,\}\ C_1\ \{\, R \,\} \qquad \{\, R \,\}\ C_2\ \{\, Q \,\}}{\{\, P \,\}\ C_1; C_2\ \{\, Q \,\}}$$

If-statement
$$\frac{\{\, P \wedge B \,\}\ C_1\ \{\, Q \,\} \qquad \{\, P \wedge \neg B \,\}\ C_2\ \{\, Q \,\}}{\{\, P \,\}\ \texttt{if}\ (B)\ \{C_1\}\ \texttt{else}\ \{C_2\}\ \{\, Q \,\}}$$

Partial-while
$$\frac{\{\, I \wedge B \,\}\ C\ \{\, I \,\}}{\{\, I \,\}\ \texttt{while}\ (B)\ \{C\}\ \{\, I \wedge \neg B \,\}}$$

Implied (consequence)
$$\frac{\vdash P \to P' \qquad \{\, P' \,\}\ C\ \{\, Q' \,\} \qquad \vdash Q' \to Q}{\{\, P \,\}\ C\ \{\, Q \,\}}$$

10.3.1 The assignment rule

The assignment rule works *backwards*: to have Q after $x = E$, we need Q with E in place of x before. It looks backwards at first but is correct; the "obvious" forward version $\{\, P \,\}\ x = E\ \{\, P[E/x] \,\}$ is wrong (try $\{\, x = 2 \,\}\ x = 3\ \{\, 3 = 2 \,\}$).

Example 10.3.

- $\{\, y + 1 = 6 \,\}\ x = y + 1\ \{\, x = 6 \,\}$, and $y + 1 = 6$ simplifies to $y = 5$.
- $\{\, x + 1 > 0 \,\}\ x = x + 1\ \{\, x > 0 \,\}$: the precondition is $x > -1$, i.e. $x \geq 0$.
- $\{\, 2 \cdot 3 = 6 \,\}\ x = 3\ \{\, 2x = 6 \,\}$: the precondition is a tautology.

10.3.2 Proof tableaux

Rather than write proof trees, we annotate the program, interleaving code and assertions. Each assertion is justified by the rule that produced it; adjacent assertions with no code between them must be related by an implication (justified by "implied", with a separate proof of that implication in ordinary maths or natural deduction). We usually fill a tableau in *from the bottom up*, pushing the postcondition backwards through assignments.

Example 10.4 (Swap with a temporary)**.**

$\{x = x_0 \wedge y = y_0\}$
$\{y = y_0 \wedge x = x_0\}$ implied
`t = x;`
$\{y = y_0 \wedge t = x_0\}$ assignment
`x = y;`
$\{x = y_0 \wedge t = x_0\}$ assignment
`y = t;`
$\{x = y_0 \wedge y = x_0\}$ assignment

Reading upward: substitute t for y, then y for x, then x for t.

Example 10.5 (Maximum)**.**

$\{\top\}$
`if (x > y) {`
  $\{\top \wedge x > y\}$ if-statement
  $\{x = \max(x, y)\}$ implied
  `m = x;`
  $\{m = \max(x, y)\}$ assignment
`} else {`
  $\{\top \wedge \neg(x > y)\}$ if-statement
  $\{y = \max(x, y)\}$ implied
  `m = y;`
  $\{m = \max(x, y)\}$ assignment
`}`
$\{m = \max(x, y)\}$ if-statement

10.4 Loops and invariants

Definition 10.6 (Loop invariant)**.** A *loop invariant* for `while (B) {C}` is a formula I such that $\models_{\text{par}} \{ I \wedge B \}\ C\ \{ I \}$: if I holds at the start of an iteration (and the guard is true), it holds again at the end.

To prove $\{ P \}$ `while (B) {C}` $\{ Q \}$ we need an I with

1. $P \to I$ (the invariant holds initially);
2. $\{ I \wedge B \}\ C\ \{ I \}$ (it is preserved by the body);
3. $I \wedge \neg B \to Q$ (on exit it gives the postcondition).

Finding invariants

- Trace the loop on a small input and tabulate variable values at the start of each iteration; look for a relationship that stays constant.
- The invariant usually "generalises" the postcondition by replacing a constant (like x or n) by the loop counter. E.g. post $y = x!$ suggests $y = z!$.
- It must be strong enough that $I \wedge \neg B \to Q$, but weak enough to be true initially and preserved. Often an extra conjunct (like $z \leq x$) is needed.

Example 10.7 (Factorial)**.** Program `Fac1`: compute $y = x!$.

$\{x \geq 0\}$	
$\{1 = 0!\}$	implied
`y = 1;`	
$\{y = 0!\}$	assignment
`z = 0;`	
$\{y = z!\}$	assignment
`while (z != x) {`	
$\quad\{y = z! \wedge z \neq x\}$	partial-while
$\quad\{y \cdot (z+1) = (z+1)!\}$	implied
`  z = z + 1;`	
$\quad\{y \cdot z = z!\}$	assignment
`  y = y * z;`	
$\quad\{y = z!\}$	assignment
`}`	
$\{y = z! \wedge \neg(z \neq x)\}$	partial-while
$\{y = x!\}$	implied

The invariant is $I \equiv y = z!$. The implied steps: $1 = 0!$ is true; $y = z! \to y(z+1) = (z+1)!$ by the definition of factorial; $y = z! \wedge z = x \to y = x!$.

Example 10.8 (Sum $1 + \cdots + n$)**.**

$\{n \geq 0\}$	
$\{0 = \sum_{k=1}^{0} k \wedge 0 \leq n\}$	implied
`s = 0; i = 0;`	
$\{s = \sum_{k=1}^{i} k \wedge i \leq n\}$	assignment (twice)
`while (i < n) {`	
$\quad\{s = \sum_{k=1}^{i} k \wedge i \leq n \wedge i < n\}$	partial-while
$\quad\{s + i + 1 = \sum_{k=1}^{i+1} k \wedge i + 1 \leq n\}$	implied
`  i = i + 1;`	
$\quad\{s + i = \sum_{k=1}^{i} k \wedge i \leq n\}$	assignment
`  s = s + i;`	
$\quad\{s = \sum_{k=1}^{i} k \wedge i \leq n\}$	assignment
`}`	
$\{s = \sum_{k=1}^{i} k \wedge i \leq n \wedge \neg(i < n)\}$	partial-while
$\{s = \sum_{k=1}^{n} k\}$	implied

Here the conjunct $i \leq n$ is essential: without it, $\neg(i < n)$ gives only $i \geq n$, not $i = n$.

10.5 Total correctness

Partial correctness ignores termination. Since only `while` loops can fail to terminate in our language, we strengthen the while rule.

Definition 10.9 (Variant)**.** A *variant* (ranking function) for a loop is an integer expression V such that

- $I \wedge B \to V \geq 0$ (it is non-negative whenever the body is about to run), and

- each execution of the body strictly decreases V.

Since a non-negative integer cannot decrease forever, the loop terminates.

$$\text{Total-while:}\quad \frac{\{\, I \wedge B \wedge 0 \le V = v_0 \,\}\ C\ \{\, I \wedge 0 \le V < v_0 \,\}}{\{\, I \wedge 0 \le V \,\}\ \texttt{while}\ (B)\ \{C\}\ \{\, I \wedge \neg B \,\}}$$

where v_0 is a fresh logical variable recording the variant's value at the start of the iteration. The other rules are the same as for partial correctness.

Example 10.10 (Total correctness of `Fac1`)**.** Use invariant $I \equiv y = z! \wedge z \le x$ and variant $V \equiv x - z$. Initially $z = 0 \le x$ by the precondition $x \ge 0$. In the body, $z \le x \wedge z \ne x$ gives $z < x$, so $V = x - z > 0$; after $z = z + 1$ the new variant is $x - z - 1 < v_0$ and still ≥ 0. Note that without the precondition $x \ge 0$ the loop runs forever on negative x, so only partial correctness would hold.

Pitfalls

- Do not forget that the invariant must hold *before* the loop, so the initialisation code is part of the proof.
- Every pair of adjacent assertions needs an "implied" justification that is actually a valid implication; check them separately.
- Variants must be integers bounded below; "x gets smaller" over the reals does not guarantee termination.

10.6 Arrays

An array assignment `a[i] = E` changes the whole array value a. We model arrays as functions and write $a\{i \leftarrow e\}$ for the array that agrees with a except at index i, where it has value e:

$$(a\{i \leftarrow e\})[j] = \begin{cases} e & \text{if } j = i, \\ a[j] & \text{if } j \ne i. \end{cases}$$

$$\text{Array assignment:}\quad \frac{}{\{\, Q[a\{i \leftarrow E\}/a] \,\}\ a[i] = E\ \{\, Q \,\}}$$

Example 10.11 (Aliasing)**.** What precondition guarantees $\{a[j] = 3\}$ after `a[i] = 3`? By the rule, $(a\{i \leftarrow 3\})[j] = 3$, which simplifies (case split on $i = j$) to $i = j \vee a[j] = 3$. Naively treating $a[i]$ like a separate variable would give just $a[j] = 3$ and miss the case where i and j are the same index.

Example 10.12 (Sum of an array)**.**

$\{n \geq 0\}$	
`s = 0; i = 0;`	
$\{s = \sum_{k=0}^{i-1} a[k] \wedge 0 \leq i \leq n\}$	assignment, implied
`while (i < n) {`	
$\quad\{s = \sum_{k=0}^{i-1} a[k] \wedge 0 \leq i \leq n \wedge i < n\}$	partial-while
$\quad\{s + a[i] = \sum_{k=0}^{i} a[k] \wedge 0 \leq i + 1 \leq n\}$	implied
`  s = s + a[i];`	
$\quad\{s = \sum_{k=0}^{i} a[k] \wedge 0 \leq i + 1 \leq n\}$	assignment
`  i = i + 1;`	
$\quad\{s = \sum_{k=0}^{i-1} a[k] \wedge 0 \leq i \leq n\}$	assignment
`}`	
$\{s = \sum_{k=0}^{n-1} a[k]\}$	partial-while, implied

With variant $n - i$ this is also totally correct. The array is only read, so the plain assignment rule suffices; the array rule is needed when the program writes to a (e.g. in-place reversal or sorting, where the invariant typically says "$a[0..i-1]$ is sorted and is a permutation of the original").

Summary

- $\models_{\text{par}} \{\, P \,\}\ C\ \{\, Q \,\}$: if P holds and C terminates, Q holds; $\models_{\text{tot}}$ additionally requires termination.
- Assignment works backwards: $\{\, Q[E/x] \,\}\ x = E\ \{\, Q \,\}$; build tableaux bottom-up.
- Loops need an invariant I: $P \to I$, $\{\, I \wedge B \,\}\ C\ \{\, I \,\}$, $I \wedge \neg B \to Q$.
- Termination: a non-negative integer variant that strictly decreases each iteration.
- Arrays: treat `a[i]=E` as assigning the updated array $a\{i \leftarrow E\}$; watch for aliasing.

Chapter 11

Undecidability and Computability

Propositional validity is decidable by truth tables, and first-order validity is semi-decidable by proof search. Is there an algorithm that decides first-order validity outright? Hilbert's *Entscheidungsproblem* (1928) asked exactly this. Church and Turing (1936) independently showed the answer is *no*. To prove such a thing we need a precise definition of "algorithm".

11.1 Turing machines

Definition 11.1 (Turing machine)**.** A (deterministic) *Turing machine* consists of

- a finite set of states Q, with a start state q_0 and halting states (e.g. q_{accept}, q_{reject});
- a tape alphabet Γ including a blank symbol $\sqcup$, and an input alphabet $\Sigma \subseteq \Gamma \setminus \{\sqcup\}$;
- a transition function $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$.

The machine has an infinite tape, initially holding the input followed by blanks, and a head on the leftmost cell. At each step, in state q reading symbol a, if $\delta(q, a) = (q', b, D)$ it writes b, moves one cell in direction D, and enters q'. It stops when it enters a halting state; it may also run forever.

Example 11.2. A machine that accepts binary strings with an even number of 1s scans right, toggling between states q_{even} and q_{odd} on each 1, and on reaching the blank accepts iff it is in q_{even}. (This one does not even need to write; machines that use the tape as memory can do far more, e.g. recognise $\{0^n 1^n : n \geq 0\}$ by repeatedly crossing off a 0 and a matching 1.)

Church–Turing thesis

Every function that is computable by an effective procedure (an "algorithm" in the intuitive sense) is computable by a Turing machine. This is not a theorem ("effective procedure" is informal) but is supported by the fact that all proposed models (lambda calculus, recursive functions, register machines, real programming languages) compute exactly the same functions.

So we may reason about "programs" in any language (say, C or Racket) instead of Turing machines. Crucially, programs are finite strings, so they can be given as *input* to other programs; a *universal* Turing machine (an interpreter) can simulate any machine given its description.

11.2 Decision problems

Definition 11.3. A *decision problem* is a yes/no question about inputs, i.e. a set L of strings (the "yes" instances).

- L is *decidable* (recursive) if some program halts on every input and answers "yes" exactly on members of L.
- L is *semi-decidable* (recursively enumerable) if some program halts with "yes" exactly on members of L (and may loop forever on non-members).

Theorem 11.4. *L is decidable iff both L and its complement are semi-decidable.*

Proof. ($\Leftarrow$) Run both semi-deciders in parallel (alternating steps); exactly one halts. ($\Rightarrow$) A decider is a semi-decider for L, and swapping its answers gives one for the complement. □

11.3 The halting problem

Definition 11.5. HALT $= \{(P, x) : \text{program } P \text{ halts on input } x\}$.

HALT is semi-decidable: simulate P on x and say "yes" if it halts.

Theorem 11.6 (Turing 1936)**.** *HALT is undecidable.*

Proof. Suppose a program `halts(P, x)` always terminates and returns true iff P halts on x. Build:

```
void diag(Program P) {
    if (halts(P, P))      // would P halt when given its own code?
        while (1) { }     // ... then loop forever
    else
        return;           // ... otherwise halt
}
```

Consider `diag(diag)`. If it halts, then `halts(diag, diag)` returned true, so `diag` enters the infinite loop and does not halt. If it does not halt, then `halts(diag, diag)` returned false, so `diag` returns, i.e. halts. Either way we have a contradiction, so `halts` cannot exist. □

This is a *diagonal argument*, the same idea as Cantor's proof that $\mathbb{R}$ is uncountable and as the liar paradox "this sentence is false".

Corollary 11.7. *The complement of HALT is not even semi-decidable.*

11.4 Reductions

Definition 11.8. A problem A *reduces* to B ($A \leq B$) if there is a computable function f with $x \in A \iff f(x) \in B$ for all x.

If $A \leq B$ and B is decidable, then A is decidable (compute $f(x)$, then run B's decider). Contrapositively:

Proving undecidability

To show B undecidable, reduce a known undecidable problem (usually HALT) *to* B: show that a decider for B would give a decider for HALT. The direction matters: reducing B to HALT proves nothing about B's hardness.

Example 11.9 (Halting on empty input)**.** $\textsc{Halt}_\varepsilon = \{P : P \text{ halts on empty input}\}$ is undecidable. Given (P, x), construct P_x: "ignore your input and run P on x". Then P_x halts on the empty input iff P halts on x. The map $(P, x) \mapsto P_x$ is computable, so $\textsc{Halt} \leq \textsc{Halt}_\varepsilon$.

Example 11.10 (Program equivalence)**.** $\textsc{Equiv} = \{(P, Q) : P, Q \text{ compute the same function}\}$ is undecidable. Reduce $\textsc{Halt}_\varepsilon$: given P, build P': "run P on empty input, then return 0" and Q: "return 0". Then $P' \equiv Q$ iff P halts on empty input.

Rice's theorem generalises these: every non-trivial property of the *function* computed by a program ("does it ever output 7?", "is it total?", "does it compute factorial?") is undecidable. So no compiler or static analyser can perfectly decide such questions; real tools are sound-but-incomplete approximations. This is also why the verification in Chapter 10 needs human-supplied invariants.

11.5 Undecidability of first-order logic

Theorem 11.11 (Church 1936, Turing 1936)**.** *The set of valid sentences of first-order logic (in a language with at least one binary predicate symbol) is undecidable.*

Proof idea. Reduce $\textsc{Halt}$ (or the Post correspondence problem) to validity. Given a machine M and input x, build a sentence $\varphi_{M,x}$ whose axioms describe M's configurations and transitions, using predicates like $\mathit{State}_q(t)$, $\mathit{Head}(t, c)$, $\mathit{Sym}_a(t, c)$ ("at time t the head is on cell c", etc.), with conclusion "at some time a halting state is reached". Then $\models \varphi_{M,x}$ iff M halts on x. A validity decider would decide $\textsc{Halt}$. □

Decidability landscape

Problem	Status
Propositional validity / satisfiability	decidable (co-NP / NP-complete)
First-order validity	semi-decidable, not decidable
First-order satisfiability	not semi-decidable (co-semi-decidable)
Monadic FOL (unary predicates only)	decidable
Presburger arithmetic ($\mathbb{N}$ with $+$)	decidable
True arithmetic ($\mathbb{N}$ with $+, \cdot$)	not even semi-decidable

11.6 Gödel's incompleteness theorems

Definition 11.12. A set of axioms Γ is

- *consistent* if it does not prove both φ and $\neg\varphi$;
- *complete* if for every sentence φ, it proves φ or proves $\neg\varphi$;
- *effectively axiomatised* if there is an algorithm that decides whether a given sentence is an axiom (e.g. PA, with its induction scheme).

(Do not confuse completeness of a *theory* with completeness of a *proof system*: Gödel proved the latter in 1929 and the failure of the former in 1931.)

Theorem 11.13 (First incompleteness theorem)**.** *Any consistent, effectively axiomatised theory that can express basic arithmetic (e.g. PA) is incomplete: there is a sentence G such that neither G nor $\neg G$ is provable. If the theory is sound for $\mathbb{N}$, then G is true in $\mathbb{N}$ but unprovable.*

Idea.

1. *Gödel numbering*: encode symbols, formulas and proofs as natural numbers, so statements about syntax become statements about numbers.
2. Show "p is the code of a proof of the formula with code n" is expressible by an arithmetic formula $\mathit{Prf}(p, n)$, and let $\mathit{Prov}(n) \equiv \exists p\, \mathit{Prf}(p, n)$.
3. *Diagonal lemma*: build a sentence G with $\vdash G \leftrightarrow \neg \mathit{Prov}(\ulcorner G \urcorner)$, i.e. G says "I am not provable".
4. If G were provable, then G would be false while provable (inconsistency, or unsoundness). So G is unprovable, and hence true.

Computability view. If true arithmetic had a complete effective axiomatisation, we could decide truth in $\mathbb{N}$ by enumerating all proofs until one of φ or $\neg\varphi$ appears. But the halting problem is expressible in arithmetic ("$\exists t$: M halts on x within t steps"), so this would decide Halt. Contradiction. This gives a quick proof of a version of the first theorem.

Theorem 11.14 (Second incompleteness theorem)**.** *A consistent, effectively axiomatised theory containing enough arithmetic cannot prove its own consistency (the sentence $\neg \mathit{Prov}(\ulcorner 0 = s(0) \urcorner)$).*

Consequences: there is no "final" axiom system for mathematics; Hilbert's programme of proving the consistency of mathematics by finitary means cannot be carried out in full. Concrete true-but-unprovable statements in PA include Goodstein's theorem and the Paris–Harrington theorem.

Summary

- Turing machines formalise "algorithm"; the Church–Turing thesis says they capture all effective computation.
- Decidable: always halts with the right answer; semi-decidable: halts on yes instances.
- The halting problem is undecidable (diagonalisation); prove other problems undecidable by reducing Halt to them.
- First-order validity is semi-decidable but undecidable (Church–Turing).
- Gödel: any consistent, effective theory of arithmetic is incomplete and cannot prove its own consistency.